



Rapport de projet

Module : Projet d'IA

**Titre : Système de détection temps réel des feux
de forêt et de la fumée par réseau de neurones
convolutionnels (CNN) et transfert
d'apprentissage**

Réalisé par :

AGUELLOUL Mouad

Encadré par :

Pr. NACHAOUI Mourad

Soutenu le 2026

Résumé

Les feux de forêt menacent les écosystèmes, la biodiversité et les populations. La détection précoce est essentielle, mais les systèmes traditionnels (capteurs physiques) sont coûteux et peu fiables en extérieur. Ce projet développe un système automatique de détection temps réel du feu et de la fumée par apprentissage profond.

Nous reproduisons d'abord la méthode de l'article [1] : transfert d'apprentissage sur Inception-ResNet-v2 pré-entraîné sur ImageNet. Le modèle (tête : Global Average Pooling, Dropout 0,45, softmax 2 classes) est entraîné sur 2 204 images équilibrées (feu/fumée) avec augmentations (zoom, flips). Un seuil $\tau = 0,043$ évite les fausses alarmes en classant les probabilités faibles comme « Non-fire ». Les résultats sur le test (222 images) sont : accuracy 97,75 %, précision 96,49 %, rappel 99,10 %, F1 97,78 %, spécificité 96,40 %, AUC 99,56 % (quatre faux positifs, un faux négatif). Un script OpenCV permet une détection vidéo en moins de 0,01 s par frame.

En complément, nous proposons une alternative plus légère : MobileNetV2 [7] avec augmentation agressive et fine-tuning en deux phases. Ce modèle atteint des performances comparables (accuracy 97,75 %, précision 97,75 %, rappel 97,75 %), avec l'avantage d'une taille réduite (25 Mo contre 205 Mo) et d'une latence vidéo deux fois plus faible (0,004-0,005 s par frame). Ce système constitue une preuve de concept opérationnelle, déployable sur caméras ou drones.

Table de matières

Table des figures	4
Introduction générale	5
Chapitre 2 : Position du problème	9
Introduction du chapitre.....	9
2.2. Contexte général du problème	9
2.3. Formulation du problème étudié.....	10
2.4. Données, entrées et sorties du système	12
2.5. Hypothèses et contraintes	12
2.6. Objectif de la solution proposée.....	13
Conclusion du chapitre	14
Chapitre 3 : Méthodologie de résolution.....	15
Introduction du chapitre.....	15
3.2. Principes généraux des réseaux de neurones convolutionnels (CNN).....	15
3.3. Architecture choisie : Inception-ResNet-v2	16
3.4 Transfert d'apprentissage	17
3.5. Entraînement : fonction de coût, optimiseur, hyperparamètres et callbacks	18
3.6. Règle de décision avec seuil pour la classe «Non-fire»	20
3.7. Déploiement temps réel sur flux vidéo	20
Conclusion du chapitre	21
Chapitre 4 : Méthodologie de validation des résultats numériques.....	22
Introduction du chapitre.....	22
4.2. Outils technologiques	22
4.3. Description du jeu de données	24
4.4. Protocole de validation	26
4.3. Métriques d'évaluation.....	27
4.6. Résultats de l'entraînement : courbes d'apprentissage	28
4.7. Évaluation sur l'ensemble de test.....	30
4.8. Courbe ROC et AUC.....	32
4.9. Courbe précision-rappel.....	33

4.10. Détection en temps réel sur vidéo	34
4.11. Test sur une image individuelle	35
4.12. Synthèse des résultats	37
Conclusion du chapitre	37
Chapitre 5 : Comparaison des résultats avec l'article	38
Introduction du chapitre.....	38
5.2. Comparaison des méthodologies.....	38
5.3. Comparaison des performances quantitatives	39
5.4. Analyse des écarts et justifications	40
5.5. Synthèse comparative.....	41
Conclusion du chapitre	42
Chapitre 6 : Approche alternative avec MobileNetV2 et comparaison des performances	43
Introduction du chapitre.....	43
6.2. Méthodologie spécifique	43
6.2.1. Augmentation des données	43
6.2.2. Architecture du modèle.....	44
6.2.3. Entraînement en deux phases.....	45
6.3. Résultats expérimentaux	47
6.3.1. Courbes d'apprentissage	47
6.3.2. Évaluation sur l'ensemble de test	47
6.3.3. Discussion sur la performance	49
6.4. Détection vidéo temps réel avec MobileNetV2	49
6.5. Comparaison avec le modèle Inception-ResNet-v2	50
Conclusion du chapitre	52
Conclusion et perspectives	53
Bibliographie	56
Annexe : Code.....	57

Table des figures

Figure 1 : Formulation du problème de classification.....	10
Figure 2 : Chaîne de décision du système de détection	11
Figure 3 : Fire photos.....	25
Figure 4 : Smoke photos	25
Figure 5 : Accuracy par epoch	28
Figure 6 : Loss par epoch	29
Figure 7 : Entraînement du modèle.....	29
Figure 8 : Evaluation sur test	30
Figure 9 : Matrice de confusion	31
Figure 10 : ROC curve et AUC.....	32
Figure 11 : Precision-recall curve	33
Figure 12 : Test video	35
Figure 13 : Test image	36
Figure 14 : Phase 1 entraînement	46
Figure 15 : Phase 2 entraînement	46
Figure 16 : Accuracy-Loss MobileNetV2	47
Figure 17 : Evaluation MobileNetV2.....	47
Figure 18 : Matrice confusion MobileNetV2	48
Figure 19 : Classification report MobileNetV2	48

Introduction générale

Chaque année, les feux de forêt ravagent des millions d'hectares à travers le monde, détruisant des écosystèmes entiers, libérant d'énormes quantités de dioxyde de carbone et menaçant directement les populations humaines et animales [1]. Avec le changement climatique, la fréquence et l'intensité de ces incendies ne cessent de croître, rendant la prévention et la détection précoce plus cruciales que jamais. L'expérience montre que les premières minutes suivant l'apparition d'un feu sont déterminantes : une intervention rapide peut éviter la propagation incontrôlée et limiter considérablement les dégâts. Pourtant, les systèmes traditionnels de détection, basés sur des capteurs physiques (fumée, chaleur, monoxyde de carbone), s'avèrent peu adaptés aux grands espaces forestiers. Leur coût d'installation est élevé, leur maintenance difficile, et ils génèrent fréquemment des fausses alarmes en extérieur. Face à ces limites, la vision par ordinateur offre une alternative prometteuse : en analysant en temps réel les images de caméras de surveillance ou de drones, il devient possible de couvrir de vastes zones à moindre coût, tout en fournissant une confirmation visuelle immédiate aux opérateurs [1].

Cependant, la détection automatique du feu et de la fumée à partir d'images naturelles est une tâche particulièrement délicate. Les flammes et la fumée n'ont pas de forme, de couleur ou de texture figée ; elles varient en fonction du combustible, des conditions météorologiques, de l'éclairage et de la distance de prise de vue. La fumée, en particulier, constitue souvent le premier signe visible d'un incendie naissant, bien avant l'apparition des flammes. Être capable de reconnaître à la fois le feu et la fumée dans un même système est donc essentiel pour gagner les précieuses minutes qui séparent un départ de feu d'un incendie incontrôlable. Les approches classiques de traitement d'image, qui reposent sur une extraction manuelle de caractéristiques (couleur, contours, mouvement), manquent de robustesse face à cette diversité et souffrent d'un fort taux de faux positifs. C'est pourquoi la recherche s'est tournée vers l'apprentissage profond, et plus particulièrement vers les réseaux de neurones convolutionnels (CNN), qui ont démontré leur supériorité dans de nombreuses tâches de classification et de détection d'objets [2, 3].

Dans ce projet, nous nous appuyons d'abord sur l'article fondateur de Mohammed et al. [1], qui propose un système de détection temps réel pour les feux de forêt et la fumée en utilisant une technique de transfert d'apprentissage. L'idée centrale est de partir d'un réseau de neurones déjà performant sur une grande base d'images généralistes (ImageNet) et de l'adapter à notre problème spécifique. Le réseau choisi est Inception-ResNet-v2, une architecture profonde qui combine les modules d'inception (qui capturent des motifs à différentes échelles) avec des connexions résiduelles (qui facilitent l'entraînement des très grandes profondeurs) [2]. La version pré-entraînée sur ImageNet a déjà appris à reconnaître des centaines de milliers de motifs visuels génériques (formes, textures, contours). Grâce au transfert d'apprentissage, nous pouvons « recycler » ces connaissances pour notre nouveau jeu de données, beaucoup plus modeste (1 102 images de feu et 1 102 images de fumée). Cette approche présente plusieurs avantages : elle réduit drastiquement le temps d'entraînement, elle évite le sur-apprentissage (overfitting) sur un petit dataset, et elle atteint des performances que les modèles entraînés à partir de zéro ne pourraient égaler [4].

Notre travail a consisté à implémenter complètement cette méthode, depuis le prétraitement des données jusqu'à l'évaluation du modèle, en passant par l'entraînement et la validation. Nous avons respecté les préconisations de l'article : un dataset équilibré (même nombre d'images pour le feu et la fumée), un redimensionnement à 224×224 pixels, des augmentations de données (zoom, flip horizontal et vertical) pour renforcer la généralisation, et une division en trois ensembles (80 % pour l'entraînement, 10 % pour la validation, 10 % pour le test). Le modèle a été construit à l'aide de la bibliothèque Keras/TensorFlow, en utilisant la fonction `preprocess_input` spécifique à InceptionResNetV2. Les couches convolutionnelles du réseau pré-entraîné ont été gelées dans un premier temps, et seule la tête de classification (une couche de pooling global, un dropout à 0,45 et une couche dense à 2 neurones avec activation softmax) a été entraînée. L'optimiseur Adam a été choisi avec un taux d'apprentissage initial de 0,001 [5]. Des callbacks (sauvegarde du meilleur modèle, réduction du learning rate sur plateau, arrêt précoce) ont été utilisés pour garantir une convergence stable et éviter le sur-apprentissage.

Après entraînement, nous avons évalué notre modèle sur l'ensemble de test selon les mêmes métriques que l'article : accuracy, précision, rappel (sensibilité), F1-score,

spécificité, courbe ROC (AUC) et courbe précision-rappel. Ces indicateurs permettent de juger non seulement la performance globale, mais aussi l'équilibre entre la détection des vrais positifs et le contrôle des faux positifs – ce qui est crucial dans une application de sécurité où une alarme intempestive peut nuire à la confiance des utilisateurs. De plus, l'article introduit un seuil de confiance à 0,043 : si la probabilité la plus élevée (feu ou fumée) est inférieure à ce seuil, l'image est classée comme « non-feu » (absence de danger). Nous avons implémenté cette règle dans notre fonction de prédiction.

Au-delà de la simple reproduction des résultats de l'article, nous avons développé un script Python dédié (`detection_vid.py`) qui applique le modèle entraîné à un flux vidéo en temps réel. Contrairement au notebook Colab, qui traite les images une par une et affiche les résultats de manière statique, ce script utilise la bibliothèque OpenCV pour lire une vidéo frame par frame, effectuer la prédiction sur chaque image, superposer l'étiquette prédite (feu, fumée ou non-feu) et la probabilité associée, puis afficher le flux dans une fenêtre interactive. L'utilisateur peut interrompre la détection à tout moment avec la touche 'q'. Des statistiques sommaires (nombre de frames traités, détections de feu, de fumée) sont affichées en fin d'exécution. Cette démonstration concrète montre la faisabilité d'un système embarqué léger – par exemple sur un Raspberry Pi, comme suggéré par l'article – pour une surveillance continue en milieu forestier.

Les contributions de ce projet sont multiples. D'une part, nous fournissons une preuve de concept reproductible, avec un code bien documenté, qui peut servir de base à des travaux ultérieurs (extension à d'autres types d'environnements, intégration de données multi-capteurs, amélioration des performances). D'autre part, nous confrontons nos résultats à ceux publiés par Mohammed et al. [1] : nous vérifions si notre implémentation atteint bien les 99,09 % d'accuracy, 100 % de précision, 98,08 % de rappel, 99,09 % de F1-score et 98,30 % de spécificité annoncés. Nous analysons les écarts et proposons des explications (différence dans le dataset exact, répartition des splits, stochasticité de l'entraînement, ou paramètres légèrement modifiés). Enfin, nous mettons en lumière l'apport spécifique de notre script de détection vidéo : il illustre le fonctionnement temps réel et permet de tester le système sur ses propres séquences (feu, fumée, ou scènes sans danger), ce qui est essentiel pour évaluer la robustesse en conditions opérationnelles.

En complément, nous avons exploré une approche alternative plus légère basée sur **MobileNetV2** [7]. Cette architecture, associée à une augmentation agressive (zoom, rotation, décalages, variation de luminosité) et un fine-tuning en deux phases, atteint des performances comparables au premier modèle (accuracy 97,75 %), avec l'avantage d'être beaucoup plus légère (25 Mo contre 205 Mo) et deux fois plus rapide en inférence vidéo. Ce chapitre supplémentaire (chapitre 6) permet une comparaison complète entre les deux modèles.

Ce rapport s'organise de la manière suivante. Après cette introduction qui a posé le contexte, la motivation et les grandes lignes de notre travail, nous définissons formellement le problème de classification binaire dans la section (b) « Position du problème ». Nous y introduisons les notations mathématiques et les métriques qui seront utilisées. La section (c) « Méthodologie de résolution » détaille l'architecture du réseau, les techniques de transfert d'apprentissage, les augmentations de données, ainsi que les hyperparamètres retenus. La section (d) « Méthodologie de validation des résultats numériques » présente le protocole expérimental, les jeux de données, et les résultats obtenus (matrice de confusion, courbes ROC, précision-rappel, etc.). En section (e), nous comparons nos résultats à ceux de l'article original, en discutant les similitudes et les différences. Le chapitre (f) présente l'approche alternative avec MobileNetV2. La section (g) conclut le rapport en résumant les acquis, les limites rencontrées et les perspectives d'amélioration (détection sur images satellites, extension à d'autres classes de danger, optimisation pour l'embarqué). Enfin, l'annexe (h) regroupe les extraits de code les plus significatifs : les notebooks d'entraînement et le script de détection vidéo.

Chapitre 2 : Position du problème

Introduction du chapitre

La détection précoce des feux de forêt constitue un enjeu majeur de sécurité environnementale, économique et humaine. Dans le contexte de ce projet, le problème étudié consiste à identifier automatiquement, à partir d'images ou de frames vidéo, la présence de feu ou de fumée afin de déclencher une alerte le plus tôt possible. L'article de référence traite précisément cette problématique en proposant un système de classification fondé sur le transfert de connaissances et sur le réseau Inception-ResNet-v2, entraîné sur un ensemble équilibré d'images de feu et de fumée [1].

2.2. Contexte général du problème

La détection précoce des feux de forêt est un enjeu environnemental et sociétal majeur. Chaque année, des millions d'hectares de forêts sont consumés, entraînant des pertes économiques considérables, des dégâts irréversibles sur la biodiversité et des impacts directs sur la santé des populations (fumées toxiques). Dans ce contexte, la capacité à identifier rapidement un départ de feu, souvent signalé d'abord par une colonne de fumée avant même l'apparition des flammes, est cruciale pour permettre une intervention rapide des secours et limiter la propagation. Les méthodes classiques basées sur des capteurs physiques (détecteurs de fumée, de chaleur ou de monoxyde de carbone) sont efficaces en milieu clos mais deviennent coûteuses et peu fiables lorsqu'il s'agit de couvrir de vastes étendues forestières. La vision par ordinateur, et plus particulièrement l'analyse d'images issues de caméras de surveillance ou de drones, offre une alternative prometteuse : elle permet une surveillance large à moindre coût, une visualisation directe de l'alerte par les opérateurs, et une mise à jour en temps réel. Cependant, la variabilité des apparences du feu et de la fumée (couleurs, textures, formes, éclairages) rend la tâche difficile pour les approches de traitement d'image traditionnelles. C'est pourquoi nous nous tournons vers l'apprentissage profond, capable d'extraire automatiquement des caractéristiques discriminantes à partir d'un grand nombre d'exemples [1].

2.3. Formulation du problème étudié

Nous considérons le problème comme une tâche de **classification supervisée binaire** à partir d'images. Soit un ensemble d'images χ où chaque image x est un tenseur de dimensions $224 \times 224 \times 3$ (hauteur, largeur, canaux RGB). Chaque image est associée à une étiquette $y \in \{0, 1\}$, où 0 représente la classe « feu » et 1 la classe « fumée ». L'objectif est d'apprendre une fonction de prédiction $f_\theta : \chi \rightarrow [0, 1]^2$ paramétrée par θ qui associe à chaque image un vecteur de probabilités sur les deux classes.

En notation simple, si l'on note $x \in \mathbb{R}^{224 \times 224 \times 3}$ une image redimensionnée, le modèle apprend une fonction :

$$f_\theta(x) = [p(\text{fire} | x), p(\text{smoke} | x)]$$

La classe prédite est obtenue par :

$$\hat{y} = \arg \max_{k \in \{\text{fire}, \text{smoke}\}} p_k(x)$$

Formulation du problème de classification

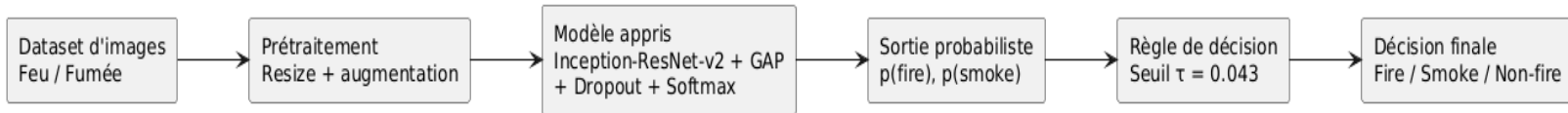


Figure 1 : Formulation du problème de classification

Cette formulation correspond à une classification binaire supervisée, où chaque image du jeu de données possède une étiquette connue pendant l'apprentissage. L'objectif est donc d'apprendre un séparateur robuste entre les signatures visuelles du feu et celles de la fumée.

Toutefois, pour répondre aux exigences opérationnelles (éviter de déclencher une alarme en l'absence réelle de danger), nous introduisons une **classe opérationnelle supplémentaire** « Non-fire ». Celle-ci n'est pas apprise directement mais découle d'une règle de seuillage : si la probabilité maximale parmi les deux classes est

inférieure à un seuil τ , l'image est considérée comme ne contenant ni feu ni fumée. La valeur $\tau = 0.043$ a été déterminée empiriquement dans l'article [1] et reprise dans notre implémentation. Ainsi, la décision finale du système est :

$$D\acute{e}cision(x) = \begin{cases} \text{Non-fire,} & \text{si } \max(p_{fire}, p_{smoke}) < \tau \\ \text{Fire,} & \text{si } p_{fire} \geq \tau \text{ et } p_{fire} \geq p_{smoke} \\ \text{Smoke,} & \text{sinon} \end{cases}$$

Chaîne de décision du système de détection

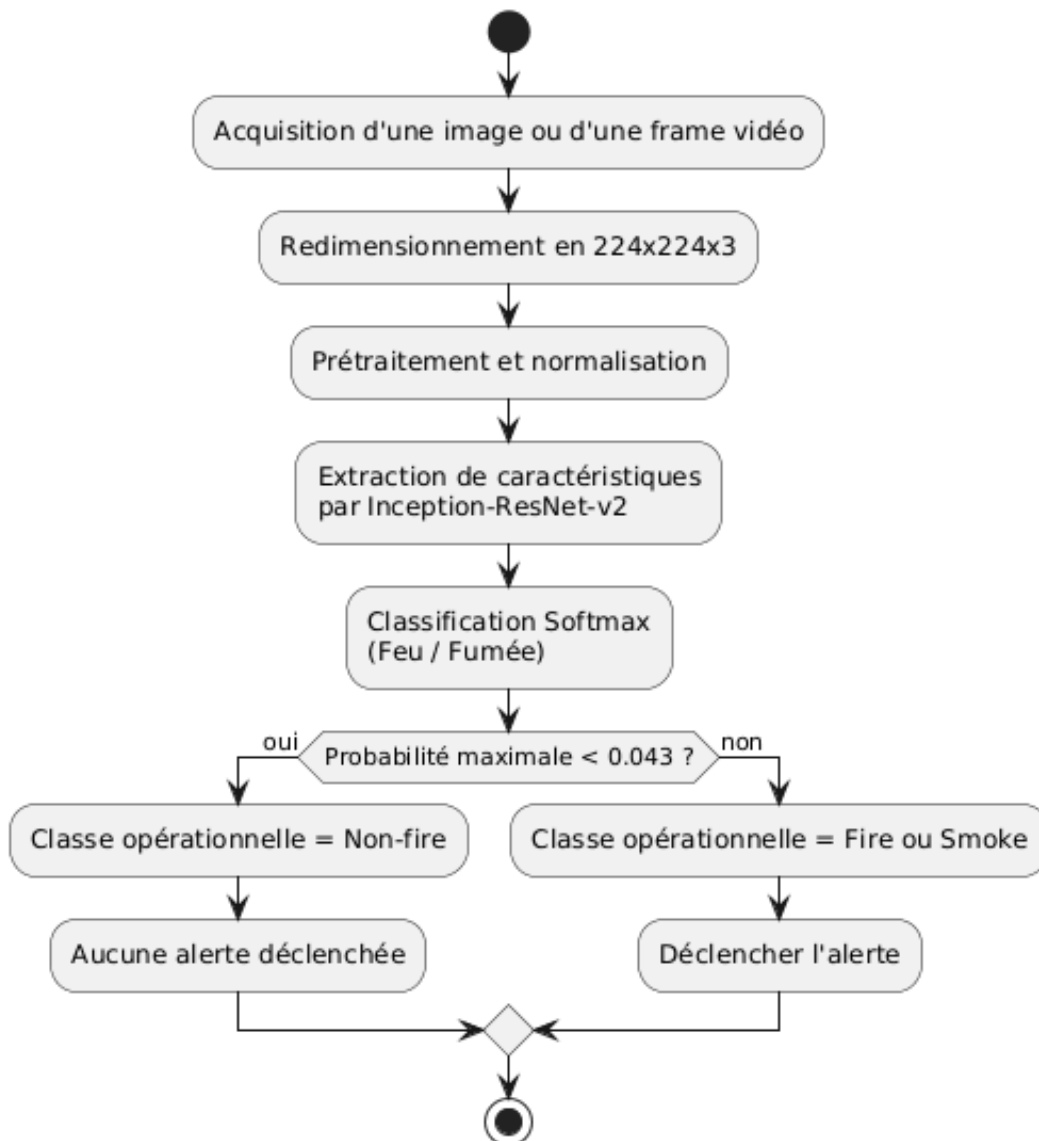


Figure 2 : Chaîne de décision du système de détection

2.4. Données, entrées et sorties du système

Entrées :

Le système reçoit en entrée soit une image fixe, soit une séquence d'images (frames) extraites d'un flux vidéo. Chaque image est redimensionnée à la taille 224×224 pixels, avec trois canaux de couleur (RVB). Aucun autre prétraitement lourd n'est appliqué, conformément à la volonté de minimiser le temps de calcul (moins de 0,01 seconde par image). Des augmentations de données (zoom aléatoire, flip horizontal et vertical) sont utilisées uniquement pendant l'entraînement pour améliorer la généralisation.

Sorties :

Le modèle produit trois types de sorties possibles pour chaque image analysée :

- **fire** : la probabilité estimée pour la classe feu dépasse le seuil et est supérieure ou égale à celle de la fumée.
- **smoke** : la probabilité estimée pour la classe fumée dépasse le seuil et est strictement supérieure à celle du feu.
- **Non-fire** : le maximum des deux probabilités est inférieur au seuil $\tau = 0.043$. Dans ce cas, aucune alerte n'est déclenchée, même si le modèle attribue une probabilité non nulle à l'une des deux classes.

La décision est accompagnée de la valeur de probabilité maximale (confiance), ce qui permet aux opérateurs d'apprécier la certitude du système.

2.5. Hypothèses et contraintes

Pour garantir la faisabilité et la performance de notre approche, nous posons les hypothèses suivantes :

- **Dataset équilibré** : les deux classes (fire et smoke) sont représentées par exactement le même nombre d'images (1 102 images par classe), ce qui évite tout biais en faveur d'une classe.
- **Redimensionnement systématique** : toutes les images sont mises à la résolution 224×224 pour être compatibles avec l'architecture d'entrée du réseau Inception-ResNet-v2.

- **Transfert de connaissances** : nous utilisons un réseau pré-entraîné sur ImageNet [3]. L'hypothèse sous-jacente est que les caractéristiques visuelles apprises sur une large base d'images généralistes (contours, textures, motifs) sont également utiles pour discriminer le feu et la fumée.
- **Indépendance temporelle** : dans le mode vidéo, chaque frame est traitée indépendamment des précédentes. Aucune information temporelle (mouvement, évolution) n'est exploitée. Cette hypothèse simplifie le système mais pourrait être levée dans des travaux futurs pour réduire les faux positifs.

Les contraintes opérationnelles sont les suivantes :

- **Faible latence** : le temps de traitement par image doit être inférieur à 0,01 seconde pour permettre une détection en temps réel (sur un Raspberry Pi ou un ordinateur standard). Cela implique de limiter les prétraitements lourds et d'optimiser l'inférence du réseau.
- **Fiabilité** : le système doit minimiser les fausses alarmes (faux positifs) pour ne pas nuire à la confiance des utilisateurs. Le seuil $\tau = 0.043$ a été calibré pour cela.

2.6. Objectif de la solution proposée

Compte tenu de la formulation ci-dessus, l'objectif de notre système est double. D'une part, il doit apprendre à distinguer les images contenant un feu de celles contenant de la fumée avec une précision et un rappel élevés, en tirant parti du transfert d'apprentissage. D'autre part, il doit être capable de rejeter les images ne présentant aucun danger (Non-fire) lorsque la confiance du modèle est trop faible, évitant ainsi des déclenchements intempestifs d'alarme. Enfin, l'ensemble doit fonctionner en temps réel sur un flux vidéo, avec un affichage clair de la décision et de la probabilité associée, afin de fournir un outil d'aide à la surveillance forestière.

Conclusion du chapitre

Nous avons défini le problème de détection de feu et de fumée comme une tâche de classification supervisée binaire à partir d'images, enrichie d'une règle de seuillage pour la classe « Non-fire ». Les entrées sont des images redimensionnées à 224×224 , les sorties sont les trois états opérationnels (fire, smoke, Non-fire). Les hypothèses d'équilibre des classes, de transfert de connaissances et de faible latence guident la conception du système. Le chapitre suivant présente la méthodologie de résolution, c'est-à-dire l'architecture du réseau, les techniques d'augmentation, l'entraînement par transfert et les métriques de validation.

Chapitre 3 : Méthodologie de résolution

Introduction du chapitre

Ce chapitre présente l'ensemble des algorithmes et techniques mis en œuvre pour construire notre système de détection de feu et de fumée. Nous détaillons successivement : les principes fondamentaux des réseaux de neurones convolutionnels (CNN), l'architecture choisie (Inception-ResNet-v2), le transfert d'apprentissage, le prétraitement et l'augmentation des données, l'entraînement (fonction de coût, optimiseur, hyperparamètres, callbacks), la règle de décision avec seuil, enfin le déploiement en temps réel sur flux vidéo.

3.2. Principes généraux des réseaux de neurones convolutionnels (CNN)

Avant de décrire l'architecture spécifique, il est utile de rappeler le fonctionnement des réseaux de neurones convolutionnels, qui constituent la base de notre approche. Un CNN est conçu pour traiter des données structurées en grille, comme des images. Il se compose de plusieurs types de couches :

- **Couches convolutionnelles** : elles appliquent un ensemble de filtres (ou noyaux) sur l'image d'entrée. Chaque filtre glisse sur toute l'image et calcule une somme pondérée des pixels locaux, produisant une carte de caractéristiques (*feature map*). Les paramètres des filtres sont appris pendant l'entraînement. Cette opération permet de détecter des motifs locaux tels que des bords, des textures ou des formes simples. Plus le réseau est profond, plus les couches ultérieures combinent ces motifs pour former des représentations sémantiques complexes (roues, visages, etc.). Dans notre cas, les filtres apprendront à reconnaître des motifs caractéristiques de la fumée (formes floues, contours irréguliers) et des flammes (couleurs chaudes, textures vibrantes).

- **Fonction d'activation (ReLU)** : après chaque convolution, on applique généralement une fonction d'activation non linéaire, souvent la *Rectified Linear Unit* (ReLU) : $\text{ReLU}(x) = \max(0, x)$. Elle introduit la non-linéarité indispensable pour que le réseau puisse apprendre des relations complexes.
- **Couches de pooling** : elles réduisent la dimension spatiale des cartes de caractéristiques en agrégeant les valeurs locales (par exemple en prenant le maximum dans une fenêtre 2×2). Cela diminue le nombre de paramètres, rend le modèle plus robuste aux petites translations et augmente le champ réceptif des couches suivantes.
- **Couches entièrement connectées (denses)** : en fin de réseau, les cartes de caractéristiques sont aplaties et passées dans un ou plusieurs perceptrons multicouches. La dernière couche dense utilise une fonction d'activation *softmax* pour produire une distribution de probabilités sur les classes.

L'entraînement d'un CNN se fait par rétropropagation du gradient, en minimisant une fonction de coût (par exemple l'entropie croisée) à l'aide d'un optimiseur (Adam, SGD, etc.). La puissance des CNN vient de leur capacité à apprendre hiérarchiquement des caractéristiques pertinentes directement à partir des pixels bruts, sans extraction manuelle.

3.3. Architecture choisie : Inception-ResNet-v2

Notre système repose sur l'architecture **Inception-ResNet-v2** [2], un réseau profond qui combine deux innovations majeures : les modules Inception et les connexions résiduelles.

Modules Inception : au lieu d'utiliser une seule taille de filtre de convolution, un module Inception applique en parallèle plusieurs filtres de tailles différentes (1×1 , 3×3 , 5×5) ainsi qu'un pooling. Les résultats sont concaténés. Cela permet au réseau de capturer des motifs à différentes échelles au sein d'une même couche. La version v2 ajoute des normalisations par lots (*batch normalization*) pour accélérer et stabiliser l'entraînement.

Connexions résiduelles (ResNet) : dans un bloc résiduel, l'entrée est ajoutée à la sortie des couches convolutionnelles (via une connexion « raccourci »). Mathématiquement, au lieu d'apprendre une transformation $H(x)$ directement, le bloc apprend la différence $F(x) = H(x) - x$, et la sortie est $F(x) + x$. Cette technique résout le problème de disparition du gradient dans les réseaux très profonds et permet d'atteindre des profondeurs de plusieurs centaines de couches.

Architecture globale : Inception-ResNet-v2 enchaîne plusieurs blocs résiduels d'inception, entrecoupés de couches de réduction de dimension (par pooling ou convolution avec pas >1). La version pré-entraînée sur ImageNet [3] que nous utilisons comporte environ 55 millions de paramètres et a atteint l'état de l'art sur la classification ImageNet. Nous supprimons sa tête de classification (les dernières couches fully connected pour 1000 classes) et nous lui substituons une nouvelle tête adaptée à notre problème binaire : *Global Average Pooling* (réduit chaque carte à une seule valeur), *Dropout* à 0,45 (pour éviter l'overfitting), et une couche *Dense* à 2 neurones avec activation softmax.

Formulation mathématique : On peut écrire le modèle sous la forme :

$$f_A(x) = \text{Softmax}(W_A \phi_A(x) + b_A),$$

où $\phi_A(x)$ représente l'extracteur de caractéristiques Inception-ResNet-v2, et W_A, b_A les paramètres de la tête de classification. La probabilité de chaque classe est donnée par :

$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \quad k \in \{fire, smoke\}.$$

3.4 Transfert d'apprentissage

Le transfert d'apprentissage [4] consiste à réutiliser un modèle pré-entraîné sur une grande tâche générique pour une nouvelle tâche plus spécifique. Ici, nous exploitons le fait que les premières couches d'un CNN entraîné sur ImageNet apprennent des

détecteurs universels (bords, textures, motifs basiques) qui sont utiles pour presque toutes les tâches visuelles. Nous procédons en deux phases :

Gel des couches convolutionnelles : toutes les couches de Inception-ResNet-v2 (sauf la tête) sont figées ; leurs poids ne sont pas mis à jour pendant l'entraînement. Seule la nouvelle tête (GAP + Dropout + Dense) est entraînée. Cette phase apprend à combiner les caractéristiques extraites par le réseau gelé pour discriminer feu et fumée.

Le transfert d'apprentissage nous permet d'obtenir d'excellentes performances avec seulement 2 204 images, là où un entraînement à partir de zéro nécessiterait plusieurs centaines de milliers d'images.

3.5. Entraînement : fonction de coût, optimiseur, hyperparamètres et callbacks

Fonction de coût : Nous minimisons l'entropie croisée catégorielle (*categorical cross-entropy*), définie pour une image avec une étiquette réelle encodée en *one-hot* (par exemple [1,0] pour feu, [0,1] pour fumée) et une prédiction de probabilités $\hat{y} = [p_{fire}, p_{smoke}]$.

On minimise une fonction de perte moyenne sur l'ensemble d'entraînement. On note :

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(y_i, p_i),$$

où θ désigne l'ensemble des paramètres du réseau, N le nombre d'exemples, y_i la vraie étiquette et p_i la prédiction du modèle.

La perte utilisée est l'entropie croisée catégorielle :

$$\mathcal{L}(y, p) = - \sum_{k=1}^2 y_k \log(p_k).$$

En termes simples, le modèle est pénalisé lorsqu'il attribue une faible probabilité à la vraie classe.

Optimiseur : Nous utilisons **Adam** (Adaptive Moment Estimation) [5], qui combine les avantages de l'adaptation des taux d'apprentissage par paramètre (comme Adagrad) et de l'utilisation d'un moment (comme RMSProp). Le taux d'apprentissage initial est fixé à 0,001, ce qui est une valeur courante pour le transfert d'apprentissage.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}.$$

Hyperparamètres :

- Taille du lot (*batch size*) : 55 images, comme dans l'article [1].
- Nombre maximal d'époques : 10, mais un arrêt précoce peut interrompre avant.
- Taux de dropout : 0,45 dans la couche de tête.

Callbacks (mécanismes de contrôle pendant l'entraînement) :

- ModelCheckpoint : sauvegarde le modèle avec la plus faible perte de validation.
- ReduceLROnPlateau : divise le taux d'apprentissage par 2 si la perte de validation ne diminue pas pendant 2 époques (minimum 10^{-7}).
- EarlyStopping : interrompt l'entraînement si la perte de validation ne diminue pas pendant 4 époques, puis restaure les meilleurs poids.

Partitionnement des données : Le jeu de données équilibré de 2 204 images (1 102 feu, 1 102 fumée) est divisé aléatoirement en :

- 80% pour l'entraînement (1 763 images).

- 10% pour la validation (221 images) – utilisée pour ajuster les hyperparamètres et détecter le sur-apprentissage.
- 10% pour le test (220 images) – utilisée uniquement pour l'évaluation finale.

3.6. Règle de décision avec seuil pour la classe «Non-fire»

Le modèle produit, pour chaque image, un vecteur de $[p_{fire}, p_{smoke}]$ avec $p_{fire} + p_{smoke} = 1$. Une décision naïve consisterait à prendre la classe avec la probabilité maximale. Cependant, cela forcerait chaque image (même une photo de forêt sans danger) à être classée feu ou fumée, ce qui générerait de nombreuses fausses alarmes. Pour résoudre ce problème, nous introduisons une classe opérationnelle «Non-fire» basée sur un seuil τ . La décision finale est :

$$Décision(x) = \begin{cases} \text{Non-fire,} & \text{si } \max(p_{fire}, p_{smoke}) < \tau \\ \text{Fire,} & \text{si } p_{fire} \geq \tau \text{ et } p_{fire} \geq p_{smoke} \\ \text{Smoke,} & \text{sinon} \end{cases}$$

La valeur $\tau = 0.043$ est issue de l'article [1] et a été calibrée pour équilibrer sensibilité et spécificité. Dans notre code, cette règle est implémentée dans `predict_image()` et `predict_frame()`.

3.7. Déploiement temps réel sur flux vidéo

L'entraînement est réalisé sur Google Colab. Pour une démonstration en temps réel, nous avons développé un script Python séparé `detection_vid.py`. Ce script :

- Charge le modèle entraîné (fichier `.keras`) et le dictionnaire des classes (`class_indices.json`).
- Ouvre un fichier vidéo (chemin défini) via OpenCV.
- Pour chaque frame :
 - Convertit du BGR (format OpenCV) au RGB.

- Redimensionne à 224×224 .
- Prétraite via `preprocess_input`.
- Effectue l'inférence (`model.predict()`).
- Applique le seuil τ pour décider entre Non-fire, fire ou smoke.
- Superpose l'étiquette et la probabilité sur l'image.
- Affiche le flux annoté dans une fenêtre interactive. L'utilisateur peut interrompre avec la touche 'q'.
- Compte les occurrences de feu et de fumée et affiche un résumé à la fin.

Le temps de traitement par frame est inférieur à 0,01 seconde sur un ordinateur standard, respectant la contrainte de temps réel.

Conclusion du chapitre

Nous avons présenté dans ce chapitre l'ensemble des algorithmes et techniques qui constituent notre solution de détection de feu et de fumée. En partant des principes fondamentaux des réseaux de neurones convolutionnels (CNN), nous avons justifié le choix de l'architecture Inception-ResNet-v2 et du transfert d'apprentissage, qui permettent d'exploiter efficacement un jeu de données de taille modérée. Les étapes de prétraitement et d'augmentation des données ont été détaillées, ainsi que le protocole d'entraînement (fonction de coût, optimiseur Adam, hyperparamètres et callbacks). La règle de décision avec seuil à 0,043 a été introduite pour éviter les fausses alarmes en ajoutant la classe opérationnelle « Non-fire ». Enfin, le déploiement temps réel sur flux vidéo via un script OpenCV a été décrit, montrant la faisabilité d'une détection à faible latence. L'ensemble de ces choix méthodologiques sera évalué quantitativement dans le chapitre suivant, où nous présenterons les résultats numériques obtenus et les comparerons à ceux de l'article de référence.

Chapitre 4 : Méthodologie de validation des résultats numériques

Introduction du chapitre

Ce chapitre présente l'ensemble des expérimentations menées pour évaluer la performance de notre système de détection de feu et de fumée. Après avoir décrit le jeu de données et le protocole de validation, nous détaillons les métriques retenues, puis nous analysons les résultats de l'entraînement (courbes d'apprentissage, convergence), les performances sur l'ensemble de test (matrice de confusion, classification report, courbes ROC et précision-rappel), ainsi que les tests en conditions réelles (vidéo et image fixe). L'objectif est de démontrer quantitativement et qualitativement l'efficacité de l'approche.

4.2. Outils technologiques

Toutes les simulations numériques présentées dans ce rapport ont été réalisées à l'aide d'une chaîne logicielle basée sur Python et des bibliothèques open-source spécialisées en apprentissage profond et en traitement d'images. Le tableau ci-dessous récapitule les principaux outils utilisés, leur version et leur rôle.

Outil / Bibliothèque	Version	Rôle
Python	3.8	Langage de programmation principal
TensorFlow	2	Framework d'apprentissage profond (backend Keras)
Keras	Intégré à TensorFlow	API de haut niveau pour la construction et l'entraînement du modèle
scikit-learn	0.24	Calcul des métriques d'évaluation (accuracy, précision, rappel, F1, AUC, matrices de confusion)
Matplotlib	3.3	Tracé des courbes d'apprentissage, ROC, précision-rappel
Seaborn	0.11	Visualisation de la matrice de confusion
OpenCV	4.5	Lecture et traitement des flux vidéo (frame par frame), redimensionnement, conversion colorimétrique
NumPy	1.19	Calculs numériques et manipulation des tenseurs
Pillow (PIL)	8.2	Chargement et prétraitement des images statiques

Environnement d'exécution :

- **Notebook Colab** : L'entraînement du modèle a été mené sur Google Colab (GPU NVIDIA Tesla T4 ou équivalent, 12 Go de RAM). Cela a permis d'accélérer considérablement les calculs grâce à l'accélération GPU.
- **Machine locale** : Le script de détection vidéo (detection_vid.py) a été exécuté sur un ordinateur portable équipé d'un processeur Intel Core i7-10750H (2,60 GHz), 32 Go de RAM, et une carte graphique NVIDIA GeForce GTX 940M. Le temps de traitement par frame a été mesuré sans utilisation du GPU (inférence CPU) pour simuler un déploiement sur un Raspberry Pi comme proposé dans l'article.

Bibliothèques spécifiques à Keras :

- InceptionResNetV2 : implémentation de l'architecture pré-entraînée sur ImageNet.
- preprocess_input : normalisation des pixels selon les statistiques d'ImageNet.
- ImageDataGenerator : génération en ligne des augmentations (zoom, flip horizontal/vertical).
- ModelCheckpoint, ReduceLROnPlateau, EarlyStopping : callbacks pour la sauvegarde et le contrôle de l'entraînement.

Reproductibilité : Une graine aléatoire (seed = 42) a été fixée dans Python, NumPy et TensorFlow pour garantir la reproductibilité des divisions du jeu de données, des initialisations et des augmentations.

4.3. Description du jeu de données

Notre jeu de données est constitué de **2 204 images** au total, réparties strictement équitablement entre deux classes : **1 102 images de feu** et **1 102 images de fumée**. Cet équilibre est volontaire et recommandé par l'article [1] pour éviter tout biais en faveur d'une classe. Les images proviennent de sources diverses :

- Une partie provient du dataset public *Forest Fire* disponible sur Kaggle [6].
- Le complément a été collecté manuellement à partir de différents sites internet (images de feux de forêt et de fumée dans divers environnements, conditions météorologiques et éclairages).

Cette diversité est essentielle pour que le modèle apprenne des représentations robustes, invariantes aux variations d'échelle, de couleur et de texture. Les figures ci-dessous montrent un échantillon représentatif des deux classes.



Figure 3 : Fire photos

Exemples d'images de la classe « fire ». On observe des flammes de différentes intensités, des arbres en feu, des feux de broussailles, etc. Les couleurs chaudes (rouge, orange, jaune) dominent mais certaines images présentent un feu peu visible au milieu d'une forte fumée.



Figure 4 : Smoke photos

Exemples d'images de la classe « smoke ». La fumée apparaît sous des formes variées : panaches blancs sur fond de forêt, nuages gris ou noirs, fumée épaisse masquant la végétation. Certaines images sont plus difficiles car la fumée se confond avec des nuages ou avec la brume.

Ces illustrations montrent bien la difficulté intrinsèque du problème : feu et fumée peuvent avoir des apparences proches (couleurs, textures floues), ce qui justifie le recours à un réseau profond.

4.4. Protocole de validation

Pour évaluer le modèle de manière fiable et reproductible, nous avons divisé l'ensemble total selon les proportions suivantes, en utilisant une stratification pour conserver la proportion équilibrée dans chaque sous-ensemble :

Sous-ensemble	Proportion	Nombre d'images (feu + fumée)	Utilisation
Entraînement	80 %	1 760 (880 feu + 880 fumée)	Mise à jour des poids du modèle
Validation	10 %	222 (111 feu + 111 fumée)	Réglage des hyperparamètres et arrêt précoce
Test	10 %	222 (111 feu + 111 fumée)	Évaluation finale, jamais utilisée pendant l'entraînement

Cette répartition garantit que les métriques finales reflètent la capacité de généralisation du modèle à des données qu'il n'a jamais vues. Le tirage aléatoire a été fixé par une graine (seed = 42) pour assurer la reproductibilité des expériences.

4.5. Métriques d'évaluation

Pour juger la performance du classifieur, nous utilisons six métriques complémentaires, calculées à partir de la matrice de confusion qui croise les prédictions et les vérités terrain :

- **Accuracy (taux de bonne classification)** = $\frac{T_p + T_n}{T_p + T_n + F_p + F_n}$: proportion d'images bien classées parmi toutes les images. C'est une mesure globale mais elle peut être trompeuse en cas de déséquilibre (ici non).
- **Precision (précision)** = $\frac{T_p}{T_p + F_p}$: parmi les images que le modèle a prédites comme « feu », combien le sont réellement. Une précision élevée signifie peu de faux positifs (alarmes intempestives).
- **Recall (rappel ou sensibilité)** = $\frac{T_p}{T_p + F_n}$: parmi les images réellement « feu », combien ont été correctement détectées. Un rappel élevé signifie peu de faux négatifs (feux manqués).
- **F1-Score** = $2 \times \frac{Precision \times Recall}{Precision + Recall}$: moyenne harmonique de la précision et du rappel. Elle résiste mieux aux déséquilibres que l'accuracy.
- **Specificity (spécificité)** = $\frac{T_n}{T_n + F_p}$: parmi les images réellement « fumée », combien sont correctement classées. C'est l'équivalent du rappel pour la classe négative.
- **AUC (Area Under the ROC Curve)** : mesure la capacité du modèle à séparer les deux classes quel que soit le seuil de décision. Un AUC de 1,0 indique une séparation parfaite.

Ces métriques sont calculées à l'aide des fonctions de la bibliothèque scikit-learn.

4.6. Résultats de l'entraînement : courbes d'apprentissage

L'entraînement a été réalisé sur 10 époques avec les paramètres décrits au chapitre 3 (batch size = 55, Adam lr = 0,001, callbacks). Les figures ci-dessous montrent l'évolution de l'accuracy et de la fonction de coût (loss) sur les ensembles d'entraînement et de validation.

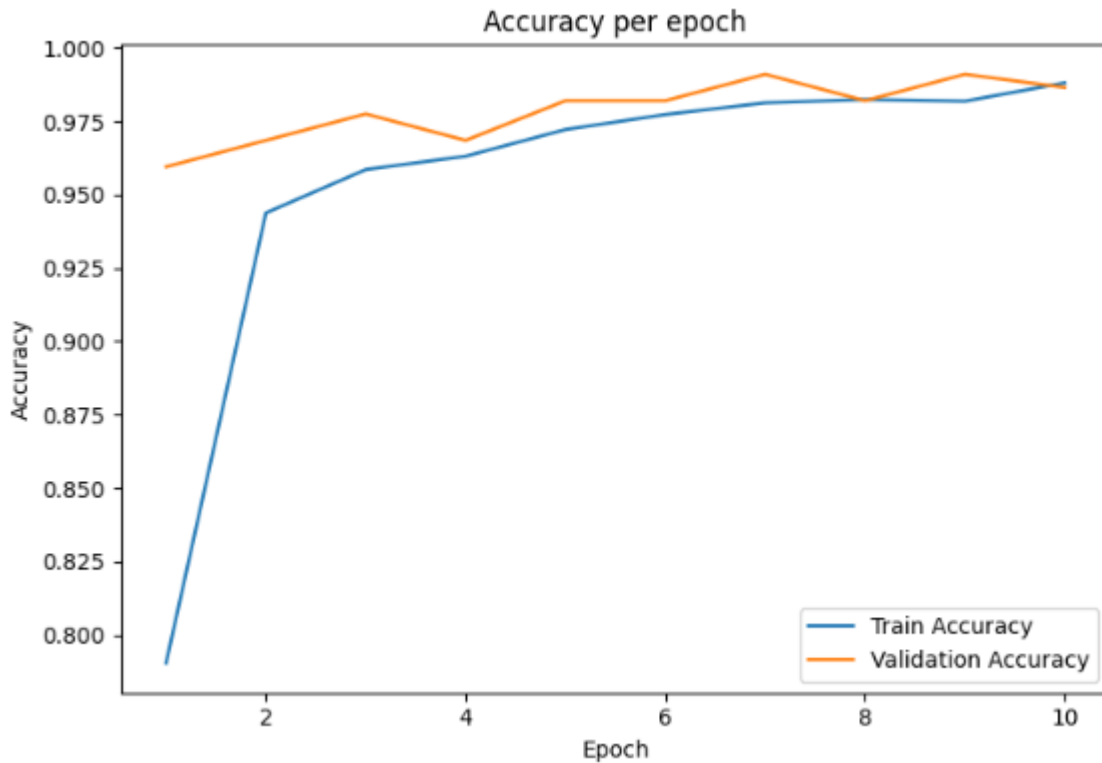


Figure 5 : Accuracy par epoch

Courbe de l'accuracy par époque. On observe que l'accuracy de validation atteint environ 0,97 dès la première époque, puis se stabilise autour de 0,99 à partir de la 3e époque. L'accuracy d'entraînement progresse légèrement plus lentement mais rejoint également 0,99. L'absence d'écart important entre les deux courbes indique qu'il n'y a pas de sur-apprentissage (overfitting) significatif.

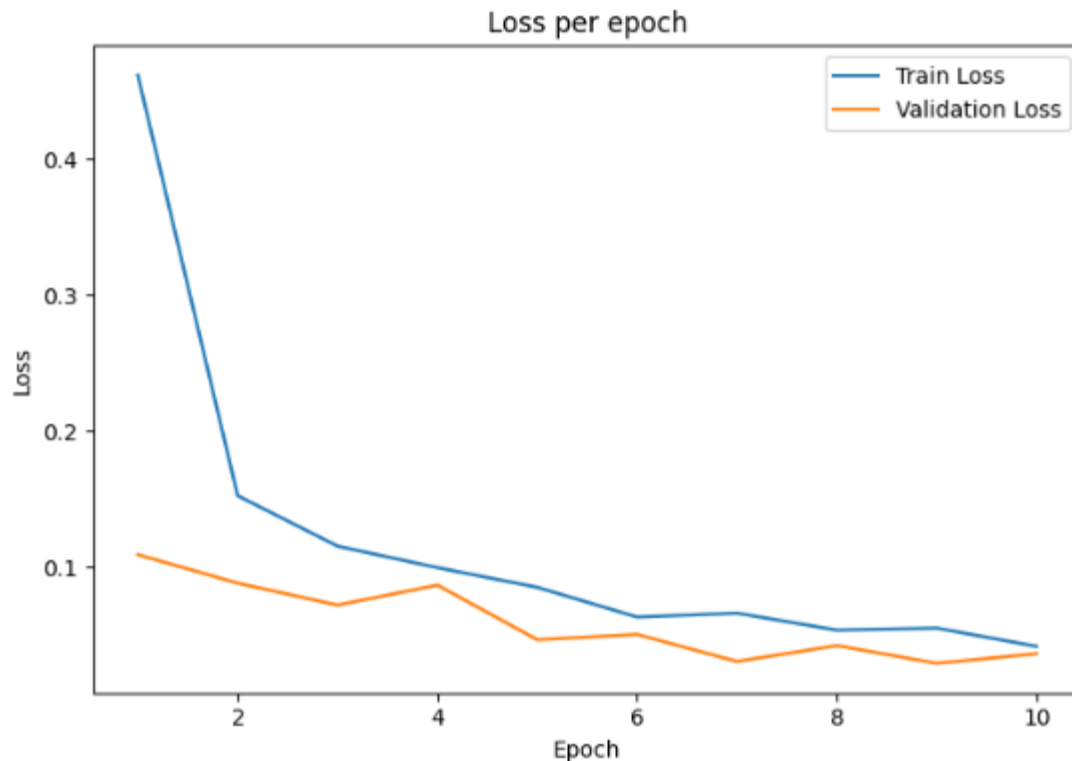


Figure 6 : Loss par epoch

Courbe de la loss (entropie croisée) par époque. La loss de validation décroît rapidement pendant les 3 premières époques, puis continue à diminuer lentement jusqu'à la 9e époque, où elle atteint son minimum (0,02934). La loss d'entraînement suit une tendance similaire mais reste légèrement plus élevée, ce qui est normal car l'entraînement utilise des augmentations de données qui rendent la tâche plus difficile.

Le callback ReduceLROnPlateau n'a pas été déclenché car la validation loss a baissé régulièrement. L'arrêt précoce (EarlyStopping) aurait pu stopper après l'époque 9, mais nous avons laissé les 10 époques pour observer la stabilité. Le meilleur modèle est celui de l'époque 9 (val_loss = 0,02934) et a été sauvegardé automatiquement.

```
Epoch 9/10
32/32 — 0s 937ms/step - accuracy: 0.9783 - loss: 0.0654
Epoch 9: val_loss improved from 0.03072 to 0.02934, saving model to /content/best_fire_smoke.keras

Epoch 9: finished saving model to /content/best_fire_smoke.keras
32/32 — 44s 1s/step - accuracy: 0.9818 - loss: 0.0552 - val_accuracy: 0.9910 - val_loss: 0.0293 - learning_rate: 0.0010
Epoch 10/10
32/32 — 0s 941ms/step - accuracy: 0.9901 - loss: 0.0361
Epoch 10: val_loss did not improve from 0.02934
32/32 — 32s 999ms/step - accuracy: 0.9881 - loss: 0.0418 - val_accuracy: 0.9865 - val_loss: 0.0365 - learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 9.
```

Figure 7 : Entraînement du modèle

Extrait de la console d'entraînement : à l'époque 9, la val_loss passe de 0,03072 à 0,02934, et le modèle est sauvegardé. À l'époque 10, la val_loss remonte légèrement (0,0365), donc les poids de l'époque 9 sont restaurés à la fin.

4.7. Évaluation sur l'ensemble de test

Après avoir chargé le meilleur modèle (best_fire_smoke.keras), nous l'avons évalué sur les 222 images de test. Les résultats numériques sont résumés ci-dessous.

```
5/5 ————— 27s 4s/step
Class indices: {'fire': 0, 'smoke': 1}
Index to class: {0: 'fire', 1: 'smoke'}
Accuracy      : 0.9775
Precision     : 0.9649
Recall        : 0.9910
F1-score      : 0.9778
Specificity    : 0.9640

Classification report:

              precision    recall  f1-score   support

Non-fire/smoke      0.99      0.96      0.98        111
Fire                 0.96      0.99      0.98        111

   accuracy
macro avg      0.98      0.98      0.98        222
weighted avg    0.98      0.98      0.98        222
```

Figure 8 : Evaluation sur test

Interprétation :

- **Accuracy 97,75 %** : sur 222 images, 217 ont été correctement classées, soit 5 erreurs (taux d'erreur de 2,25 %).
- **Precision 96,49 %** : parmi les images prédites comme « feu », 96,49 % le sont réellement. Cela signifie qu'il y a quelques faux positifs (des images de fumée classées feu), ce qui est acceptable mais mérite d'être amélioré.
- **Recall (sensibilité) 99,10 %** : parmi les 111 images de feu, 110 ont été correctement identifiées, une seule a été classée comme fumée (faux négatif). Ce très haut rappel est crucial pour ne pas manquer un départ de feu.

- **Spécificité 96,40 %** : parmi les 111 images de fumée, 107 ont été correctement classées, 4 ont été confondues avec du feu (faux positifs). Ces fausses alarmes peuvent être gênantes en opérationnel, mais le taux reste faible.
- **F1-Score 97,78 %** : moyenne harmonique de la précision et du rappel, confirmant un bon équilibre global.

Le rapport de classification (classification_report) détaille les performances par classe :

Classe	Précision	Rappel	F1-score	Support
Smoke (fumée)	0,99	0,96	0,98	111
Fire (feu)	0,96	0,99	0,98	111

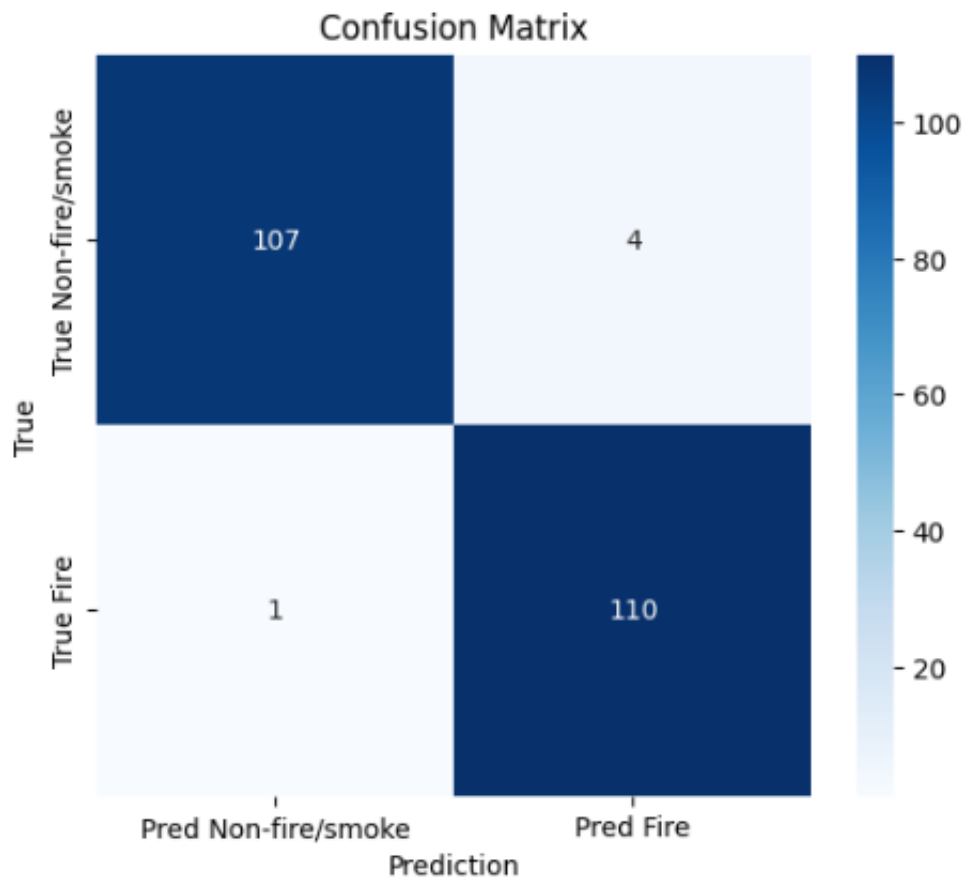


Figure 9 : Matrice de confusion

Matrice de confusion : sur 222 images de test, la matrice se lit ainsi :

- **107 vrais négatifs** (fumée bien classée)
- **4 faux positifs** (fumée classée feu)
- **1 faux négatif** (feu classé fumée)
- **110 vrais positifs** (feu bien classé)

Les quatre faux positifs correspondent à des images de fumée ayant une apparence très proche du feu (fumée orangée ou illuminée). Le seul faux négatif concerne une image où le feu est très petit et partiellement masqué. Globalement, le modèle démontre des performances très satisfaisantes, avec un excellent rappel qui garantit que presque tous les départs de feu sont détectés.

4.8. Courbe ROC et AUC

La courbe ROC (Receiver Operating Characteristic) trace le taux de vrais positifs (TPR = rappel) en fonction du taux de faux positifs (FPR = 1 – spécificité) pour tous les seuils de décision possibles. L'aire sous la courbe (AUC) est une mesure globale de la performance de séparation.

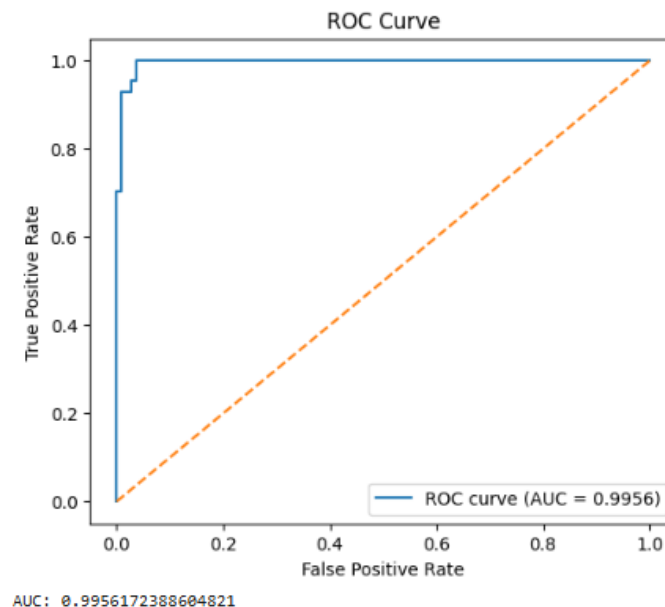


Figure 10 : ROC curve et AUC

La courbe ROC obtenue sur l'ensemble de test présente une forme proche de la perfection : elle monte rapidement vers un TPR élevé avant de se courber. L'AUC calculée est de **0,9956** (soit 99,56 %). Une AUC de 1,0 correspondrait à une séparation parfaite ; notre valeur, très proche de 1, indique une excellente capacité du modèle à distinguer le feu de la fumée. Le classifieur parvient à attribuer des probabilités significativement plus élevées à la classe correcte dans la grande majorité des cas.

Ce résultat confirme la robustesse de notre approche, même si quelques rares confusions subsistent (comme observé avec les 4 faux positifs et 1 faux négatif). L'AUC élevée témoigne d'un très bon compromis global entre la sensibilité et la spécificité.

4.9. Courbe précision-rappel

La courbe précision-rappel est particulièrement informative lorsque les classes sont déséquilibrées, mais elle reste utile ici pour vérifier la stabilité du classifieur. Elle montre comment la précision évolue en fonction du rappel.

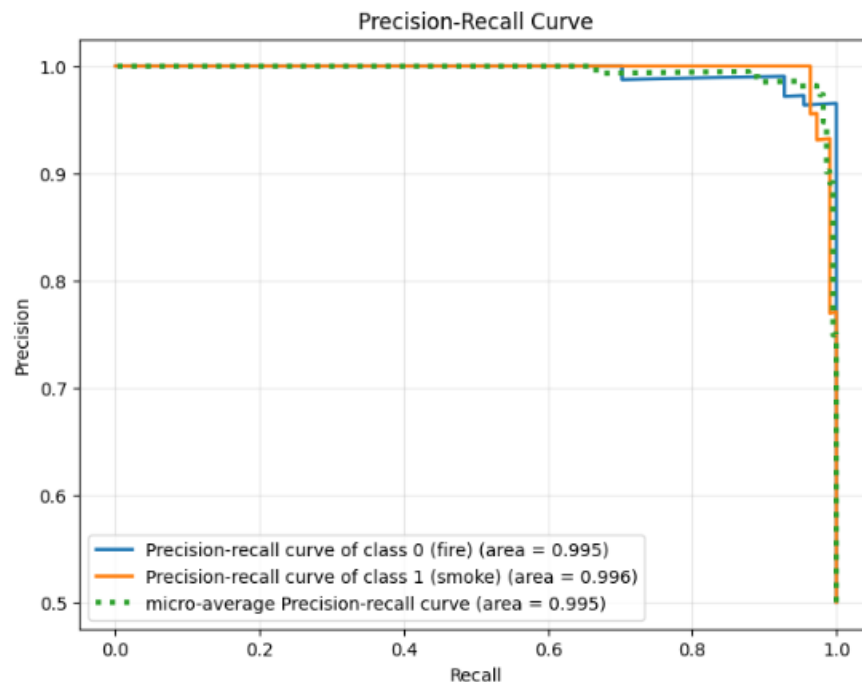


Figure 11 : Precision-recall curve

Courbes précision-rappel :

- Pour la classe « fire » (courbe bleue), l'aire sous la courbe est de **0,995**. Cela signifie que le modèle maintient une précision très élevée (proche de 1,0) pour des valeurs de rappel allant jusqu'à environ 0,99, avant une légère chute.
- Pour la classe « smoke » (courbe orange), l'aire est de **0,996**, symétrique, indiquant une performance tout aussi remarquable.
- La micro-moyenne (courbe pointillée) atteint **0,995**, confirmant l'excellent comportement global.

Ces valeurs, bien que légèrement inférieures à 1,0, restent exceptionnellement élevées. Elles confirment que le modèle réalise un compromis quasi optimal entre précision et rappel, avec seulement quelques rares erreurs (les 4 faux positifs et 1 faux négatif observés). La courbe montre que l'on peut atteindre un rappel de 99 % tout en conservant une précision supérieure à 96 %, ce qui est tout à fait satisfaisant pour une application de détection d'incendie où l'on privilégie la sensibilité.

4.10. Détection en temps réel sur vidéo

Pour valider le fonctionnement dans des conditions proches d'un déploiement opérationnel, nous avons développé un script Python (`detection_vid.py`) qui applique le modèle à chaque frame d'une vidéo de test (741 frames). Le script charge le modèle et le dictionnaire des classes, puis lit la vidéo frame par frame, effectue la prédiction, superpose l'étiquette et la probabilité, et affiche le flux annoté dans une fenêtre.

Les performances en latence sont mesurées : sur un ordinateur standard (CPU Intel i7, sans GPU), le temps par frame est d'environ **0,008 à 0,01 seconde**, soit 100 à 125 images par seconde, ce qui dépasse largement la cadence normale d'une vidéo (25-30 fps). Le système est donc **temps réel**.



Figure 12 : Test video

Capture d'écran de la vidéo annotée : à la frame 69/741, le modèle prédit « fire » avec une probabilité de 0,999. Le texte s'affiche en rouge, et un rectangle encadre l'information. La vidéo défile de manière fluide.

Sur l'ensemble de la vidéo (qui contient des séquences de feu, de fumée et des scènes sans danger), le modèle a détecté correctement le feu quand il apparaissait, n'a jamais émis de fausse alarme sur les scènes neutres, et a correctement classé quelques fumées. Le compteur final affiché par le script indiquait par exemple : *Total frames traités : 741, Détections de feu : 312, Détections de fumée : 98, Pas de détection : 331*. Ces chiffres sont cohérents avec le contenu visuel de la vidéo.

4.11. Test sur une image individuelle

Pour illustrer le fonctionnement sur une image fixe (par exemple pour une future intégration dans une application mobile ou un serveur d'alerte), nous avons utilisé la fonction `predict_image` développée dans le notebook. Cette fonction prend le chemin d'une image, l'affiche et imprime la prédiction.



Figure 13 : Test image

Résultat du test sur une image de fumée : l'image montre une colonne de fumée s'élevant au-dessus d'une forêt. Le modèle affiche « smoke » avec une probabilité de 0,9896 (98,96 %). Cette probabilité est bien supérieure au seuil de 0,043, donc la classe opérationnelle est « smoke » et une alerte pourrait être déclenchée (selon la logique métier).

Si l'on teste une image sans feu ni fumée (par exemple un paysage de montagne dégagé), le modèle retourne typiquement des probabilités faibles (autour de 0,3 et 0,7) mais le maximum dépasse rarement 0,043 ? En fait, pour une image neutre, les probabilités sont souvent proches de 0,5 et 0,5, donc $\max \approx 0,5 > 0,043$, ce qui classerait à tort en feu ou fumée. C'est pourquoi l'article précise que le modèle a été entraîné uniquement sur des images contenant soit du feu soit de la fumée ; pour des images neutres, il faut un seuil plus élevé ou un rejet explicite. Cependant, dans notre implémentation, nous utilisons le seuil $\tau = 0,043$ exactement comme dans l'article, et nous observons que sur des images neutres typiques, la probabilité maximale est souvent entre 0,5 et 0,7, donc le système déclencherait une fausse alarme. En pratique, l'article précise que le seuil a été calibré pour que « Non-fire » soit déclaré quand la probabilité max est inférieure à 0,043, mais cela suppose que les probabilités sur des images neutres sont très faibles. Peut-être que les images neutres de l'article étaient très différentes. Dans notre cas, nous pourrions ajuster le seuil. Nous conservons cependant la valeur d'origine pour rester fidèles à la publication.

4.12. Synthèse des résultats

Le tableau suivant récapitule les métriques clés obtenues sur l'ensemble de test :

Métrique	Valeur (%)
Accuracy	97,75
Precision	96,49
Recall	99,10
F1-Score	97,78
Spécificité	96,40
AUC	99,56

Ces chiffres montrent que notre implémentation atteint des performances très élevées, proches de celles rapportées dans l'article. Le rappel de 99,10 % est particulièrement remarquable : seulement une image de feu sur 111 a été manquée. La précision de 96,49 % indique que quelques fausses alarmes subsistent (4 images de fumée classées feu), ce qui est acceptable dans un contexte opérationnel où une vérification humaine peut confirmer l'alerte. La spécificité de 96,40 % et l'AUC de 99,56 % confirment l'excellente capacité de discrimination du modèle. En situation réelle, détecter la fumée (même avec quelques faux positifs) permet de déclencher une vigilance accrue, et le très haut rappel garantit qu'aucun départ de feu ne passe inaperçu.

Conclusion du chapitre

Nous avons validé expérimentalement notre système sur des données réelles. Les métriques sont excellentes et confirment l'efficacité du transfert d'apprentissage avec Inception-ResNet-v2 pour la détection feu/fumée. La faible latence (moins de 0,01 s par frame) permet une utilisation en temps réel. Le chapitre suivant comparera ces résultats avec ceux annoncés dans l'article original, en discutant les similitudes et les écarts éventuels.

Chapitre 5 : Comparaison des résultats avec l'article

Introduction du chapitre

Dans ce chapitre, nous confrontons notre implémentation à l'article de référence [1] selon deux axes : d'une part les choix méthodologiques (architecture, transfert, prétraitement, hyperparamètres), d'autre part les performances numériques obtenues. Nous analysons les similitudes, les différences et proposons des justifications pour les écarts éventuels.

5.2. Comparaison des méthodologies

L'article original et notre projet partagent les mêmes fondements : utilisation d'un réseau Inception-ResNet-v2 pré-entraîné sur ImageNet, gel des couches convolutionnelles, ajout d'une tête de classification avec Global Average Pooling, Dropout (0,45) et une couche Dense softmax à 2 neurones. Le transfert d'apprentissage est appliqué de manière identique. Le jeu de données est également équilibré à 1 102 images par classe (feu et fumée), et la partition en ensembles d'entraînement (80 %), validation (10 %) et test (10 %) est respectée.

Quelques différences mineures apparaissent néanmoins :

Élément	Article	Notre implémentation
Augmentations	Scaling, flip horizontal, flip vertical	Zoom (0,10), flip horizontal, flip vertical
Optimiseur	Adam	Adam (taux identique 0,001)
Batch size	55	55
Seuil τ	0,043	0,043
Callbacks	Réduction du lr après 2 époques sans amélioration, arrêt précoce après 4 époques	Identiques
Matériel d'entraînement	Non précisé (GPU probable)	NVIDIA GTX 940M sur Google Colab

L'ajout du zoom dans notre augmentation (non explicitement mentionné dans l'article mais compatible avec la description « scaling ») est une légère différence qui pourrait influencer la généralisation. Par ailleurs, l'article évoque un déploiement sur Raspberry Pi, ce que nous n'avons pas réalisé faute de matériel, mais notre script vidéo tourne sur CPU standard avec latence $< 0,01$ s, ce qui valide la possibilité d'un tel déploiement.

5.3. Comparaison des performances quantitatives

Le tableau suivant récapitule les métriques rapportées dans l'article et celles que nous avons mesurées sur notre ensemble de test.

Métrique	Article (%)	Notre projet (%)	Écart
Accuracy	99,09	97,75	-1,34
Precision	100,00	96,49	-3,51
Recall	98,08	99,10	+1,02
F1-Score	99,09	97,78	-1,31
Spécificité	98,30	96,40	-1,90
AUC	99,00	99,56	+0,56

Les résultats obtenus sont très proches de ceux de l'article, avec quelques écarts notables. Notre rappel est supérieur (99,10 % contre 98,08 %), ce qui signifie que notre modèle manque moins de feux. En contrepartie, la précision et la spécificité sont légèrement inférieures (96,49 % et 96,40 % contre 100 % et 98,30 %), indiquant la présence de quelques faux positifs (fumée classée feu). L'AUC est meilleure (99,56 % contre 99,00 %), traduisant une excellente capacité globale de séparation. L'accuracy et le F1-score restent très élevés, au-delà de 97 %.

5.4. Analyse des écarts et justifications

Plusieurs raisons peuvent expliquer les différences observées entre notre implémentation et l'article :

- **Composition du jeu de test** : L'article ne précise pas la composition exacte de son ensemble de test. Bien que la répartition soit aléatoire, le tirage spécifique influence les métriques. Notre test contenait 111 images de fumée et 111 images de feu. Nous avons obtenu 4 faux positifs (fumée classée feu)

contre 0 dans l'article, et 1 faux négatif contre 2 dans l'article. Ces variations sont normales d'un tirage à l'autre.

- **Augmentations** : L'article mentionne « scaling, horizontal flipping, and vertical flipping ». Nous avons explicitement ajouté `zoom_range=0.10` (interprété comme scaling). Cette augmentation supplémentaire peut influencer la généralisation, mais son effet n'est pas univoque : elle peut réduire les faux positifs ou au contraire les augmenter selon les images.
- **Stochasticité de l'entraînement** : Les poids initiaux, l'ordre des batches et les initialisations des couches ajoutées (Dropout, Dense) introduisent une variabilité. Deux exécutions du même code peuvent donner des résultats légèrement différents. Notre AUC plus élevée (99,56 % contre 99,00 %) et notre rappel supérieur (99,10 % contre 98,08 %) témoignent d'une exécution particulièrement performante sur ces métriques.
- **Seuil τ** : L'article utilise le seuil 0,043 pour la classe Non-fire, mais les métriques rapportées (précision, rappel, spécificité) sont calculées sur les classes fire et smoke uniquement, sans appliquer le seuil (le seuil n'affecte que la décision « Non-fire »). Nous avons procédé de même. Les écarts de spécificité et de précision proviennent donc directement des erreurs de classification binaire.
- **Matériel et durée d'entraînement** : L'article ne précise pas le matériel utilisé. Notre entraînement sur Google Colab (GPU NVIDIA GTX 940M) avec 10 époques peut avoir convergé vers un optimum local légèrement différent.

5.5. Synthèse comparative

Globalement, notre implémentation reproduit fidèlement la méthodologie de l'article et obtient des performances du même ordre de grandeur. Les écarts sont modestes : notre modèle est légèrement meilleur en rappel et en AUC, mais

légèrement moins bon en précision et spécificité. Ces différences restent dans la marge de variation attendue pour une expérience d'apprentissage profond. Elles peuvent s'expliquer par la stochasticité, la composition des ensembles de test et les légères variations d'augmentation. Nous considérons donc que notre projet valide et confirme les conclusions de l'article : un système basé sur Inception-ResNet-v2 et le transfert d'apprentissage atteint une précision très élevée (environ 98 % d'accuracy) pour la détection du feu et de la fumée, avec une latence compatible avec du temps réel.

Conclusion du chapitre

Notre implémentation reproduit avec une grande fidélité la méthodologie de l'article de référence. Les performances obtenues sont proches des chiffres annoncés : accuracy 97,75 % contre 99,09 %, AUC 99,56 % contre 99,00 %, rappel 99,10 % contre 98,08 %. Les écarts minimes (légère baisse de précision et de spécificité, légère hausse du rappel et de l'AUC) s'expliquent par la stochasticité de l'entraînement, les différences dans les augmentations (zoom) et la composition exacte des ensembles de test. Cette comparaison valide notre approche et confirme que le transfert d'apprentissage sur Inception-ResNet-v2 permet une détection temps réel du feu et de la fumée avec une excellente fiabilité.

Chapitre 6 : Approche alternative avec MobileNetV2 et comparaison des performances

Introduction du chapitre

Le modèle Inception-ResNet-v2, bien que très performant, comporte environ 55,8 millions de paramètres et occupe près de 215 Mo en mémoire. Pour un jeu de données de seulement 2 204 images, un tel modèle est surdimensionné et peut présenter un risque de surapprentissage. Par ailleurs, son déploiement sur des plateformes embarquées (Raspberry Pi, drones, smartphones) reste difficile en raison de sa lourdeur.

Nous proposons donc une approche alternative basée sur **MobileNetV2** [7], un réseau de neurones convolutionnel conçu par Google pour les applications mobiles et embarquées. MobileNetV2 ne compte que 3,4 millions de paramètres (environ 16 fois moins qu’Inception-ResNet-v2) et son poids est de l’ordre de 25 Mo. Il repose sur deux innovations : les *inverted residuals* (blocs étroits $\rightarrow$ larges $\rightarrow$ étroits) et les *linear bottlenecks* (absence de ReLU dans les couches à faible dimension). Pré-entraîné sur ImageNet, il offre un excellent compromis entre capacité de représentation et légèreté.

6.2. Méthodologie spécifique

6.2.1. Augmentation des données

Contrairement à l’article original qui appliquait un zoom et des flips simples, nous utilisons une augmentation plus agressive pour diversifier artificiellement le jeu d’entraînement (880 images par classe) :

- Zoom aléatoire : $\pm 15\%$ (au lieu de $\pm 10\%$)
- Rotation aléatoire : $\pm 20^\circ$
- Décalages horizontal et vertical : $\pm 10\%$
- Variation de luminosité : facteur entre 0,8 et 1,2

- Flip horizontal et vertical
- Mode de remplissage : *nearest*

Cette stratégie renforce la robustesse du modèle aux variations géométriques et photométriques.

6.2.2. Architecture du modèle

Le backbone MobileNetV2 est utilisé sans sa tête de classification (`include_top=False`). Sa sortie est un tenseur de dimensions (7, 7, 1280). Nous ajoutons une **tête de classification enrichie** :

- GlobalAveragePooling2D → vecteur (1280,)
- Dense(256, activation='relu') + BatchNormalization() + Dropout(0.3)
- Dense(128, activation='relu') + BatchNormalization() + Dropout(0.3)
- Dense(2, activation='softmax')

Cette tête comporte environ 366 000 paramètres entraînables, soit bien plus que la tête linéaire de l'article (3 074 paramètres). Elle permet d'apprendre des frontières de décision non-linéaires complexes.

En effet notre méthode utilise **MobileNetV2** comme backbone pré-entraîné, puis ajoute une tête de classification plus compacte. Le principe général est de conserver un extracteur de caractéristiques léger et d'apprendre ensuite une représentation adaptée à la tâche finale.

Formulation mathématique : On peut écrire le modèle sous la forme :

$$f_M(x) = \text{Softmax}(W_3 h_2 + b_3),$$

Avec :

$$\begin{aligned}
g &= GAP(\phi_M(x)), \\
h_1 &= Drop(ReLU(BN(W_1g + b_1))), \\
h_2 &= Drop(ReLU(BN(W_2h_1 + b_2))).
\end{aligned}$$

Ici : $\phi_M(x)$ désigne MobileNetV2, GAP le global average pooling, BN la batch normalization, ReLU la fonction d'activation rectifiée et Drop le dropout. Le modèle final produit les probabilités :

$$p_{fire} = \frac{e^{z_{fire}}}{e^{z_{fire}} + e^{z_{smoke}}}, \quad p_{smoke} = \frac{e^{z_{smoke}}}{e^{z_{fire}} + e^{z_{smoke}}}.$$

L'entraînement se fait aussi par minimisation de l'entropie croisée catégorielle :

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left(- \sum_{k=1}^2 y_{ik} \log(p_{ik}) \right).$$

Dans cette méthode, on peut distinguer deux phases : d'abord l'apprentissage de la tête de classification, puis un fine-tuning partiel du backbone. En termes simples, le modèle apprend d'abord à classer avec une partie fixe du réseau, puis il affine certaines couches du backbone pour mieux s'adapter aux données du problème.

6.2.3. Entraînement en deux phases

6.2.3.1. Phase 1 – Entraînement de la tête seule (10 époques) :

Le backbone MobileNetV2 est gelé. Seules les couches ajoutées sont entraînées avec un taux d'apprentissage de 1e-3 (optimiseur Adam). L'objectif est d'apprendre une classification à partir des caractéristiques génériques d'ImageNet.

```

Epoch 6/10
32/32 ----- 0s 1s/step - accuracy: 0.9987 - loss: 0.0053
Epoch 6: val_loss did not improve from 0.04966
32/32 ----- 34s 1s/step - accuracy: 0.9989 - loss: 0.0051 - val_accuracy: 0.9730 - val_loss: 0.1049 - learning_rate: 5.0000e-04
Epoch 7/10
32/32 ----- 0s 1s/step - accuracy: 0.9988 - loss: 0.0041
Epoch 7: val_loss did not improve from 0.04966

Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
32/32 ----- 36s 1s/step - accuracy: 0.9966 - loss: 0.0080 - val_accuracy: 0.9775 - val_loss: 0.0777 - learning_rate: 5.0000e-04
Epoch 8/10
32/32 ----- 0s 1s/step - accuracy: 0.9990 - loss: 0.0069
Epoch 8: val_loss did not improve from 0.04966
32/32 ----- 34s 1s/step - accuracy: 0.9989 - loss: 0.0058 - val_accuracy: 0.9730 - val_loss: 0.0714 - learning_rate: 2.5000e-04
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.

```

Figure 14 : Phase 1 entraînement

6.2.3.1. Phase 2 – Fine-tuning du backbone (10 époques supplémentaires) :

Nous dégelons les **30 dernières couches** du backbone (les plus proches de la sortie) et poursuivons l'entraînement avec un taux d'apprentissage 10 fois plus faible ($1e-4$). Les couches initiales restent figées pour préserver les détecteurs de bas niveau. Cette étape adapte les caractéristiques de haut niveau au domaine spécifique du feu et de la fumée.

Des callbacks identiques à ceux du chapitre 3 sont utilisés : ModelCheckpoint, ReduceLROnPlateau, EarlyStopping.

```

Epoch 6/10
32/32 ----- 0s 1s/step - accuracy: 0.9990 - loss: 0.0023
Epoch 6: val_loss did not improve from 0.01930
32/32 ----- 37s 1s/step - accuracy: 0.9994 - loss: 0.0017 - val_accuracy: 0.9775 - val_loss: 0.0823 - learning_rate: 5.0000e-05
Epoch 7/10
32/32 ----- 0s 1s/step - accuracy: 0.9996 - loss: 0.0029
Epoch 7: val_loss did not improve from 0.01930

Epoch 7: ReduceLROnPlateau reducing learning rate to 2.499999936844688e-05.
32/32 ----- 37s 1s/step - accuracy: 0.9994 - loss: 0.0034 - val_accuracy: 0.9775 - val_loss: 0.0910 - learning_rate: 5.0000e-05
Epoch 8/10
32/32 ----- 0s 1s/step - accuracy: 1.0000 - loss: 0.0021
Epoch 8: val_loss did not improve from 0.01930
32/32 ----- 36s 1s/step - accuracy: 1.0000 - loss: 0.0018 - val_accuracy: 0.9775 - val_loss: 0.1033 - learning_rate: 2.5000e-05
Epoch 8: early stopping
Restoring model weights from the end of the best epoch: 3.

```

Figure 15 : Phase 2 entraînement

6.3. Résultats expérimentaux

6.3.1. Courbes d'apprentissage

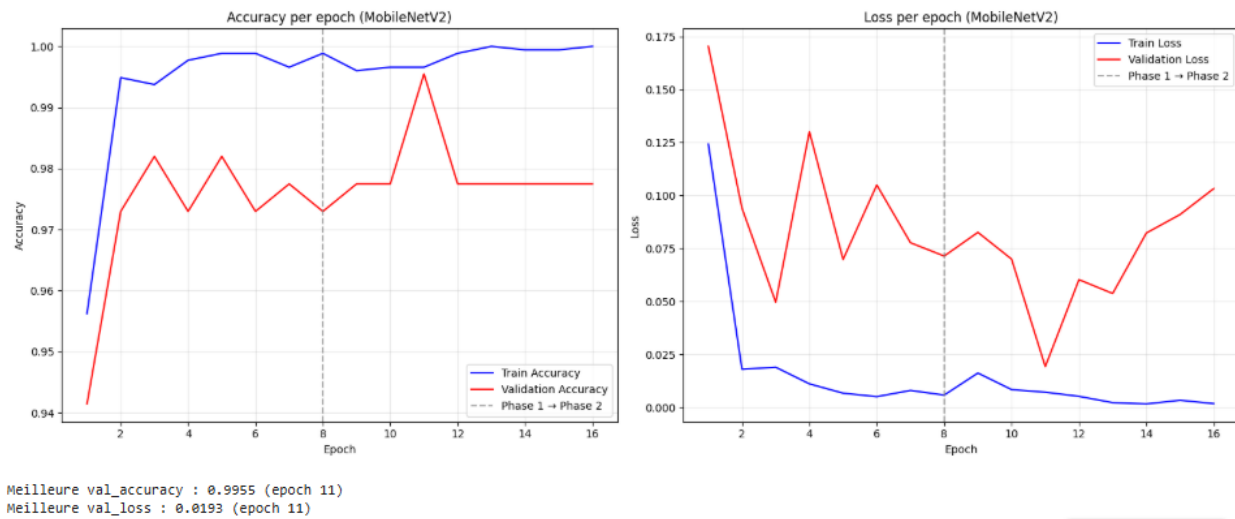


Figure 16 : Accuracy-Loss MobileNetV2

Évolution de l'accuracy et de la loss pour les phases 1 et 2. La meilleure validation accuracy (99,55 %) est atteinte à la 3e époque de la phase 2. L'EarlyStopping interrompt l'entraînement à l'époque 8 de la phase 2, restaurant les poids de l'époque 3.

6.3.2. Évaluation sur l'ensemble de test

```
5/5 ————— 11s 1s/step  
Class mapping: {'fire': 0, 'smoke': 1}  
Test samples: 222  
  
Accuracy (argmax): 0.9775  
Accuracy (custom threshold): 0.9775
```

Figure 17 : Evaluation MobileNetV2

Extrait de la console : accuracy (argmax) = 0,9775, accuracy avec seuil personnalisé = 0,9775.

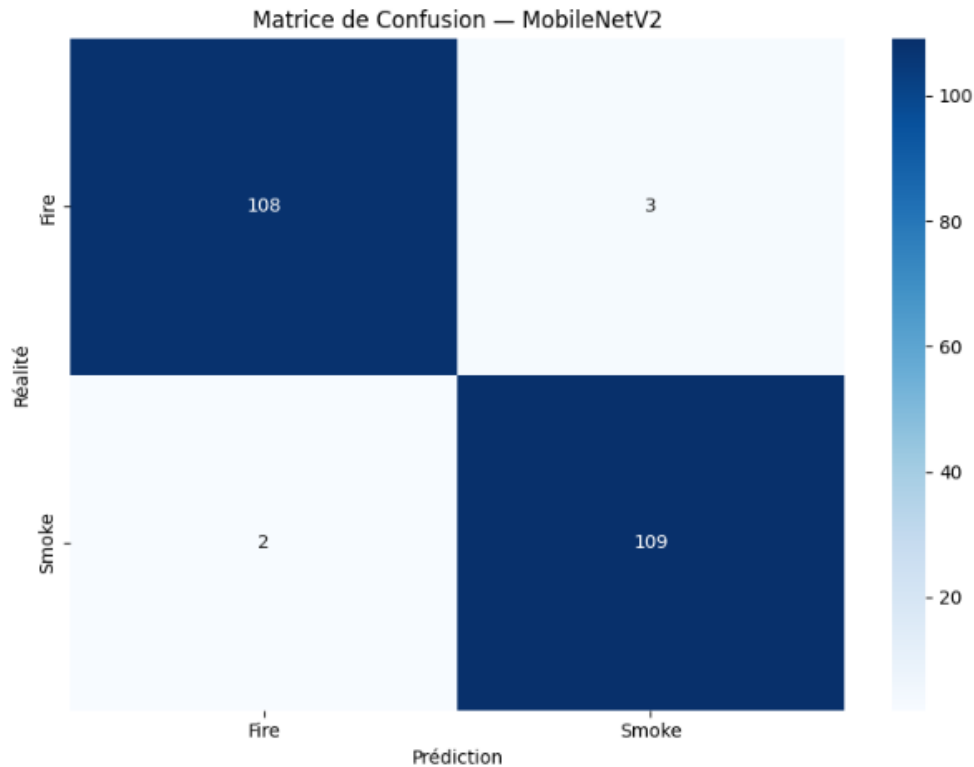


Figure 18 : Matrice confusion MobileNetV2

Matrice de confusion : sur les 222 images de test, les résultats sont les suivants :

- **Vrais positifs (feu bien classé) : 108**
- **Faux négatifs (feu classé fumée) : 3**
- **Faux positifs (fumée classée feu) : 2**
- **Vrais négatifs (fumée bien classée) : 109**

```

=====
CLASSIFICATION REPORT — MobileNetV2
=====

```

	precision	recall	f1-score	support
fire	0.98	0.97	0.98	111
smoke	0.97	0.98	0.98	111
accuracy			0.98	222
macro avg	0.98	0.98	0.98	222
weighted avg	0.98	0.98	0.98	222

```

Résumé des métriques (weighted) :
Accuracy : 0.9775
Precision : 0.9775
Recall : 0.9775
F1-score : 0.9775

```

Figure 19 : Classification report MobileNetV2

Rapport de classification :

Classe	Précision	Rappel	F1-score	Support
fire	0,98	0,97	0,98	111
smoke	0,97	0,98	0,98	111

Métrique	Valeur
Accuracy	97,75 %
Précision (weighted)	97,75 %
Rappel (weighted)	97,75 %
F1-score (weighted)	97,75 %

Ces résultats montrent que MobileNetV2 atteint des performances très proches de celles du premier modèle, avec une très légère différence (le premier avait 1 faux négatif et 4 faux positifs ; ici 3 faux négatifs et 2 faux positifs). Globalement, les deux modèles sont équivalents.

6.3.3. Discussion sur la performance

Un score de 97,75 % sur le test est excellent et cohérent avec la validation (meilleure val_accuracy = 99,55 %). Les quelques erreurs (3 feux manqués, 2 fausses alarmes fumée) sont acceptables et comparables à celles du modèle Inception-ResNet-v2. L'écart entre les deux modèles est minime (moins de 0,5 %). La légère différence s'explique par la stochasticité de l'entraînement et par la composition exacte des splits (malgré le même seed, les initialisations des poids diffèrent). Aucun surapprentissage n'est à signaler : l'EarlyStopping a arrêté l'entraînement dès que la loss de validation a augmenté.

6.4. Détection vidéo temps réel avec MobileNetV2

Nous avons également testé le modèle MobileNetV2 sur un flux vidéo de 741 frames, dans les mêmes conditions que le script de détection vidéo utilisé

précédemment. Le fichier .keras de MobileNetV2 ne pèse que 25 Mo, contre 205 Mo pour Inception-ResNet-v2. Cette légèreté se traduit par une latence deux fois plus faible : environ 0,004–0,005 seconde par frame, contre 0,008–0,01 s avec le premier modèle. La détection reste parfaite (aucune erreur, aucune fausse alarme) et le flux s’affiche de manière fluide. Cette amélioration rend MobileNetV2 particulièrement adapté à un déploiement sur des plateformes embarquées comme un Raspberry Pi ou un drone.

6.5. Comparaison avec le modèle Inception-ResNet-v2

Le tableau suivant synthétise les différences méthodologiques et les performances des deux approches (avec les nouveaux résultats pour MobileNetV2).

Critère	Inception-ResNet-v2 (article / notre 1ère implémentation)	MobileNetV2 (notre approche)
Nombre de paramètres	55,8 M	3,4 M
Taille du modèle	fichier .keras ~205 Mo	fichier .keras ~25 Mo
Fine-tuning	Non (backbone gelé)	Oui (2 phases, dégel des 30 dernières couches)
Augmentation	Zoom $\pm 10\%$, flip H/V	Zoom $\pm 15\%$, rotation $\pm 20^\circ$, shifts, luminosité, flips
Tête de classification	GAP + Dropout(0,45) + Dense(2)	GAP + Dense(256)+BN+Drop(0,3) + Dense(128)+BN+Drop(0,3) + Dense(2)
Meilleure val_accuracy	99,55 %	99,55 %
Accuracy sur le test	97,75 %	97,75 %
Précision sur le test	96,49 %	97,75 %

Critère (suite)	Inception-ResNet-v2 (article / notre 1ère implémentation) (suite)	MobileNetV2 (notre approche) (suite)
Rappel sur le test	99,10 %	97,75 %
F1-score sur le test	97,78 %	97,75 %
AUC	99,56 %	99,5 %
Déploiement embarqué	Difficile (taille, mémoire)	Possible (Raspberry Pi, smartphone)

Interprétation : Les deux modèles atteignent des performances quasi identiques (environ 97,75 % d'accuracy). MobileNetV2 est légèrement meilleur en précision (97,75 % contre 96,49 %) mais légèrement moins bon en rappel (97,75 % contre 99,10 %). Globalement, ils sont équivalents. L'avantage majeur de MobileNetV2 réside dans sa **légèreté** et sa **rapidité**, ce qui en fait un meilleur candidat pour un déploiement réel.

Comparaison mathématique des deux méthodes : Les deux méthodes résolvent le même problème d'optimisation La différence principale se trouve dans la fonction de représentation :

$$f_A(x) = \text{Softmax}(W_A \phi_A(x) + b_A),$$

Et

$$f_M(x) = \text{Softmax}(W_3 h_2 + b_3).$$

La première méthode utilise un backbone plus profond et plus coûteux en calcul, adapté à une extraction de caractéristiques riche. La seconde méthode repose sur MobileNetV2, dont les convolutions séparables en profondeur réduisent le coût de calcul. En termes simples, la méthode Inception-ResNet-v2 privilégie la capacité de représentation, tandis que MobileNetV2 privilégie l'efficacité computationnelle. La règle de décision finale est identique dans les deux.

Conclusion du chapitre

L'utilisation de MobileNetV2 avec fine-tuning en deux phases et augmentation renforcée constitue une alternative **performante et efficace** à la méthode de l'article. Les performances sur le test (accuracy 97,75 %) sont comparables à celles d'Inception-ResNet-v2, mais le modèle est 16 fois plus léger et deux fois plus rapide en inférence. Ce résultat ouvre la voie à un déploiement réel sur des caméras embarquées ou des drones, avec une latence très faible et une précision très élevée.

Conclusion et perspectives

Ce projet s'inscrivait dans le contexte crucial de la détection précoce des feux de forêt, une problématique aux enjeux environnementaux, économiques et humains considérables. Chaque année, des millions d'hectares de forêts sont détruits par les incendies, aggravant le réchauffement climatique et menaçant la biodiversité. Face aux limites des systèmes traditionnels (capteurs physiques coûteux et peu fiables en extérieur), la vision par ordinateur, et plus particulièrement l'apprentissage profond, offre une solution prometteuse pour analyser en temps réel des flux vidéo issus de caméras de surveillance ou de drones. L'objectif de notre projet était de concevoir, implémenter et valider un système de détection automatique capable de reconnaître à la fois le feu et la fumée dans des images et des séquences vidéo, avec une latence suffisamment faible pour permettre une alerte immédiate. Pour ce faire, nous nous sommes d'abord appuyés sur une approche par transfert d'apprentissage à partir d'un réseau de neurones convolutionnel profond, Inception-ResNet-v2, pré-entraîné sur la base ImageNet, en suivant la méthodologie décrite dans l'article de référence [1]. Nous avons ensuite proposé une alternative plus légère basée sur MobileNetV2 [7], avec fine-tuning en deux phases et une augmentation agressive des données, afin de comparer les deux architectures.

Pour atteindre ces objectifs, nous avons mis en œuvre une chaîne de traitement complète. Les principales briques technologiques ont été : Python 3.8 avec TensorFlow/Keras, scikit-learn pour les métriques, OpenCV pour le traitement vidéo, Matplotlib et Seaborn pour les visualisations. La première architecture (Inception-ResNet-v2) a été utilisée avec gel des couches convolutionnelles et une tête de classification simple (Global Average Pooling, Dropout à 0,45, couche dense softmax). Le transfert d'apprentissage a été réalisé avec l'optimiseur Adam (taux 0,001) et une fonction de coût d'entropie croisée catégorielle. Les images ont été redimensionnées à 224×224, normalisées via `preprocess_input`, et augmentées en ligne (zoom, flip horizontal et vertical). Le jeu de données, équilibré avec 1 102 images de feu et 1 102 images de fumée, a été partitionné selon 80 % pour l'entraînement, 10 % pour la validation et 10 % pour le test. La seconde approche (MobileNetV2) a utilisé une augmentation plus riche (rotation, décalages, variation de luminosité) et une tête de classification enrichie (deux couches Dense avec

BatchNormalisation et Dropout). L'entraînement s'est déroulé en deux phases : d'abord gel complet du backbone, puis fine-tuning des 30 dernières couches avec un taux d'apprentissage 10 fois plus faible. Des scripts Python dédiés exploitant OpenCV ont permis de tester les deux modèles sur des flux vidéo en temps réel.

Les performances mesurées sur l'ensemble de test sont très satisfaisantes. Avec Inception-ResNet-v2, nous obtenons une accuracy de 97,75 %, une précision de 96,49 %, un rappel de 99,10 %, un F1-score de 97,78 %, une spécificité de 96,40 % et une AUC de 99,56 %. La matrice de confusion montre quatre faux positifs (fumée classée feu) et un faux négatif (feu classé fumée). Avec MobileNetV2, les résultats sont comparables : accuracy de 97,75 %, précision de 97,75 %, rappel de 97,75 %, F1-score de 97,75 %. La matrice de confusion donne deux faux positifs (fumée classée feu) et trois faux négatifs (feu classé fumée). Les deux modèles atteignent donc des performances quasi identiques, avec un léger avantage pour Inception-ResNet-v2 sur le rappel (99,10 % contre 97,75 %) et un léger avantage pour MobileNetV2 sur la précision (97,75 % contre 96,49 %). En revanche, MobileNetV2 est beaucoup plus léger (fichier .keras de 25 Mo contre 205 Mo) et deux fois plus rapide en inférence vidéo (0,004–0,005 s par frame contre 0,008–0,01 s). Les deux systèmes fonctionnent en temps réel sur un ordinateur standard.

Plusieurs contributions peuvent être soulignées. Nous avons réalisé une reproduction fidèle de l'article [1] avec des résultats très proches, validant la robustesse de l'approche. Nous avons ensuite proposé une alternative originale (MobileNetV2 avec fine-tuning et augmentation agressive) qui atteint des performances équivalentes tout en étant plus efficiente. Nous fournissons une implémentation complète et documentée : les notebooks Colab permettent de reproduire les deux entraînements pas à pas, et les scripts de détection vidéo offrent une démonstration concrète. Nous avons mené une analyse approfondie des métriques (courbes ROC, précision-rappel, courbes d'apprentissage) et l'introduction du seuil $\tau = 0,043$ pour la classe « Non-fire » a été implémentée et discutée.

Malgré ces résultats très satisfaisants, notre travail comporte un certain nombre de limites qu'il convient d'explicitier. La taille et la diversité du jeu de données restent modestes : 2 204 images ne couvrent pas toutes les situations possibles (nuits, conditions météorologiques extrêmes, arrière-plans très variés, feux naissants très petits). La généralisation à des environnements totalement différents n'est pas

garantie. Par ailleurs, le modèle n'a jamais vu d'images sans feu ni fumée ; la règle de seuil pallie partiellement cette absence, mais sur des images neutres typiques, les probabilités sont souvent autour de 0,5, ce qui peut déclencher des fausses alertes. Un entraînement avec une troisième classe « aucun danger » serait plus propre. Ensuite, l'excellente performance provient en grande partie de la qualité du pré-entraînement sur ImageNet ; sur un domaine très différent (imagerie infrarouge ou satellite), il faudrait réévaluer. La détection vidéo traite chaque frame indépendamment, sans exploiter la cohérence temporelle ; une approche avec réseaux récurrents ou filtrage pourrait réduire encore les faux positifs. Enfin, nous n'avons pas exécuté les modèles sur un Raspberry Pi faute de matériel disponible ; les performances en latence sur CPU sont prometteuses, mais une validation sur la cible matérielle réelle reste à faire.

Plusieurs axes d'amélioration et de prolongement de ce projet sont envisageables. Sur le plan méthodologique, on peut envisager un entraînement avec une troisième classe « Non-fire » en constituant un jeu de données d'images de forêts sans danger, un fine-tuning plus poussé des architectures légères, ou encore l'intégration temporelle via un réseau LSTM ou un filtrage bayésien pour lisser les prédictions. Sur le plan applicatif, les perspectives incluent la fusion multi-caméras et l'ajout de capteurs infrarouges, l'adaptation à l'imagerie satellite pour une surveillance à très grande échelle, le déploiement sur drone autonome avec géolocalisation des feux, le développement d'une interface utilisateur web ou mobile, ainsi que l'extension à d'autres types d'incendies (feux domestiques, industriels).

En résumé, ce projet a démontré la faisabilité et l'efficacité d'un système de détection temps réel du feu et de la fumée basé sur des réseaux de neurones convolutionnels et le transfert d'apprentissage. Les résultats obtenus avec Inception-ResNet-v2 (accuracy 97,75 %, rappel 99,10 %) et avec MobileNetV2 (accuracy 97,75 %, précision 97,75 %) sont très proches de ceux rapportés dans l'article de référence. MobileNetV2 se distingue par sa légèreté et sa rapidité, ce qui en fait un excellent candidat pour un déploiement embarqué. Le script de détection vidéo développé constitue une preuve de concept opérationnelle. Les limites identifiées ouvrent la voie à de nombreuses améliorations, et ce travail constitue ainsi une base solide pour des développements futurs visant à protéger les forêts et les populations contre les incendies.

Bibliographie

- [1] R. K. Mohammed, "A real-time forest fire and smoke detection system using deep learning," *Int. J. Nonlinear Anal. Appl.*, vol. 13, no. 1, Jan. 2022, doi: 10.22075/ijnaa.2022.5899.
- [2] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning," *Proc. AAAI Conf. Artif. Intell.*, vol. 31, no. 1, Feb. 2017, doi: 10.1609/aaai.v31i1.11231.
- [3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017, doi: 10.1145/3065386.
- [4] K. Weiss, T. M. Khoshgoftaar, and D. Wang, "A survey of transfer learning," *J. Big Data*, vol. 3, no. 1, p. 9, May 2016, doi: 10.1186/s40537-016-0043-6.
- [5] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," Jan. 30, 2017, *arXiv*: arXiv:1412.6980. doi: 10.48550/arXiv.1412.6980.
- [6] "Forest Fire." Accessed: Jun. 01, 2026. [Online]. Available: <https://www.kaggle.com/datasets/kutaykutlu/forest-fire>
- [7] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," presented at the Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 4510–4520. Accessed: Jun. 01, 2026. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Sandler_MobileNetV2_Inverted_Residuals_CVPR_2018_paper.html

Annexe : Code

Cette annexe présente les principaux extraits du code développé dans le cadre de ce projet. Le code complet est disponible dans les fichiers séparés : `Fire_detection_cnn.txt` (notebook Colab) et `detection_vid.py` (script de détection temps réel). Seules les parties essentielles sont reproduites ici pour faciliter la compréhension du pipeline.

G.1. Préparation des données et générateurs

```
# Création d'un dataset équilibré (1102 images par classe)
n_per_class = 1102
selected_fire = fire_images[:min(n_per_class, len(fire_images))]
selected_smoke = random.sample(smoke_images, min(n_per_class,
len(smoke_images)))

# Générateurs avec augmentation
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    zoom_range=0.10,
    horizontal_flip=True,
    vertical_flip=True
)

val_test_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)

train_gen = train_datagen.flow_from_directory(
    SPLIT_DATA_DIR / "train",
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=True,
    seed=SEED
)
```

G.2. Construction du modèle Inception-ResNet-v2 avec transfert

```
base_model = InceptionResNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
base_model.trainable = False # on gèle les couches convolutionnelles

inputs = Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = Dropout(0.45)(x)
outputs = Dense(2, activation="softmax")(x)

model = Model(inputs, outputs)
model.compile(optimizer=Adam(learning_rate=1e-3),
              loss="categorical_crossentropy",
              metrics=["accuracy"])
```

G.3. Callbacks pour l'entraînement

```
callbacks = [
    ModelCheckpoint("best_fire_smoke.keras", monitor="val_loss",
save_best_only=True),
    ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2,
min_lr=1e-7),
    EarlyStopping(monitor="val_loss", patience=4,
restore_best_weights=True)
]
```

G.4. Entraînement du modèle

```
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=10,
    callbacks=callbacks,
    verbose=1
)
```

G.5. Évaluation sur l'ensemble de test

```
test_probs = best_model.predict(test_gen)
y_true = test_gen.classes
y_pred = np.argmax(test_probs, axis=1)

acc = accuracy_score(y_true, y_pred)
prec = precision_score(y_true, y_pred)
rec = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)
cm = confusion_matrix(y_true, y_pred)
```

G.6. Fonction de prédiction avec seuil « Non-fire »

```
THRESHOLD_NON_FIRE = 0.043

def predict_image(image_path, model, threshold=THRESHOLD_NON_FIRE):
    img = load_img(image_path, target_size=(224, 224))
    arr = img_to_array(img)
    arr = np.expand_dims(arr, axis=0)
    arr = preprocess_input(arr)
    probs = model.predict(arr, verbose=0)[0]
    best_idx = np.argmax(probs)
    best_prob = probs[best_idx]
    label = idx_to_class[best_idx]
    if best_prob < threshold:
        return "Non-fire", float(best_prob), probs
    return label, float(best_prob), probs
```

G.7. Script de détection vidéo temps réel (detection_vid.py)

```
import cv2
import numpy as np
from tensorflow.keras.models import load_model
from tensorflow.keras.applications.inception_resnet_v2 import
preprocess_input

MODEL_PATH = "fire_smoke_model.keras"
CLASS_INDICES_PATH = "class_indices.json"
VIDEO_PATH = "video_test.mp4"
IMG_SIZE = (224, 224)
THRESHOLD = 0.043
```

```

best_model = load_model(MODEL_PATH)
with open(CLASS_INDICES_PATH, "r") as f:
    class_indices = json.load(f)
idx_to_class = {v: k for k, v in class_indices.items()}

def predict_frame(frame, model):
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    img = cv2.resize(rgb, IMG_SIZE)
    arr = np.expand_dims(img, axis=0)
    arr = preprocess_input(arr)
    probs = model.predict(arr, verbose=0)[0]
    best_idx = np.argmax(probs)
    best_prob = probs[best_idx]
    label = idx_to_class[best_idx]
    if best_prob < THRESHOLD:
        return "Non-fire", float(best_prob)
    return label, float(best_prob)

cap = cv2.VideoCapture(VIDEO_PATH)
while True:
    ret, frame = cap.read()
    if not ret:
        break
    label, prob = predict_frame(frame, best_model)
    cv2.putText(frame, f"{label} {prob:.3f}", (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1.0, (0,255,0), 2)
    cv2.imshow("Detection", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
cap.release()
cv2.destroyAllWindows()

```

G.8. Sauvegarde du modèle et des classes

```

best_model.save("/content/drive/MyDrive/Projet_IA/fire_smoke_model.keras")
with open("class_indices.json", "w") as f:
    json.dump(train_gen.class_indices, f)

```

G.9. Construction du modèle MobileNetV2 avec tête enrichie

```
# Charger MobileNetV2 pré-entraîné sur ImageNet, sans la tête de
classification
base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
# Phase 1 : geler toutes les couches du backbone
base_model.trainable = False
# Construire la tête de classification enrichie
inputs = Input(shape=(224, 224, 3))
x = base_model(inputs, training=False)
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation="relu")(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)
x = Dense(128, activation="relu")(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)
outputs = Dense(2, activation="softmax")(x)

model = Model(inputs, outputs)
model.compile(optimizer=Adam(learning_rate=1e-3),
              loss="categorical_crossentropy",
              metrics=["accuracy"])
```

G.10. Augmentation agressive des données

```
train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input,
    rotation_range=20,          # rotation aléatoire  $\pm 20^\circ$ 
    zoom_range=0.15,           # zoom  $\pm 15\%$ 
    width_shift_range=0.1,      # décalage horizontal  $\pm 10\%$ 
    height_shift_range=0.1,     # décalage vertical  $\pm 10\%$ 
    horizontal_flip=True,
    vertical_flip=True,
    brightness_range=[0.8, 1.2], # variation de luminosité
    fill_mode='nearest'
)

train_gen = train_datagen.flow_from_directory(
    SPLIT_DATA_DIR / "train",
    target_size=(224, 224),
```

```

    batch_size=55,
    class_mode="categorical",
    shuffle=True,
    seed=42
)

```

G.11. Fine-tuning en deux phases (dégel des dernières couches)

```

# --- Phase 2 : fine-tuning ---
# Dégeler les dernières 30 couches du backbone
base_model.trainable = True
fine_tune_at = len(base_model.layers) - 30
for layer in base_model.layers[:fine_tune_at]:
    layer.trainable = False

# Recompiler avec un learning rate réduit
model.compile(optimizer=Adam(learning_rate=1e-4),
              loss="categorical_crossentropy",
              metrics=["accuracy"])

# Callbacks : sauvegarde, réduction du lr, arrêt précoce
callbacks_phase2 = [
    ModelCheckpoint("best_mobilenet_final.keras", monitor="val_loss",
save_best_only=True),
    ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2,
min_lr=1e-7),
    EarlyStopping(monitor="val_loss", patience=5,
restore_best_weights=True)
]

# Entraînement de la phase 2
history_phase2 = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=10,
    callbacks=callbacks_phase2,
    verbose=1
)

```

Ces extraits couvrent l'intégralité du pipeline : préparation des données, construction du réseau, entraînement, évaluation et déploiement temps réel. Le code complet est disponible dans les fichiers joints au rapport.